เอกสารประกอบการสอน

วิชา โครงสรางระบบคอมพิวเตอร์

(Computer System Organization)

บทที่ 5 เรื่องไมโครโปรเซสเซอร์และการเชื่อมต่อระบบบัส

(Microprocessor and Bus System)

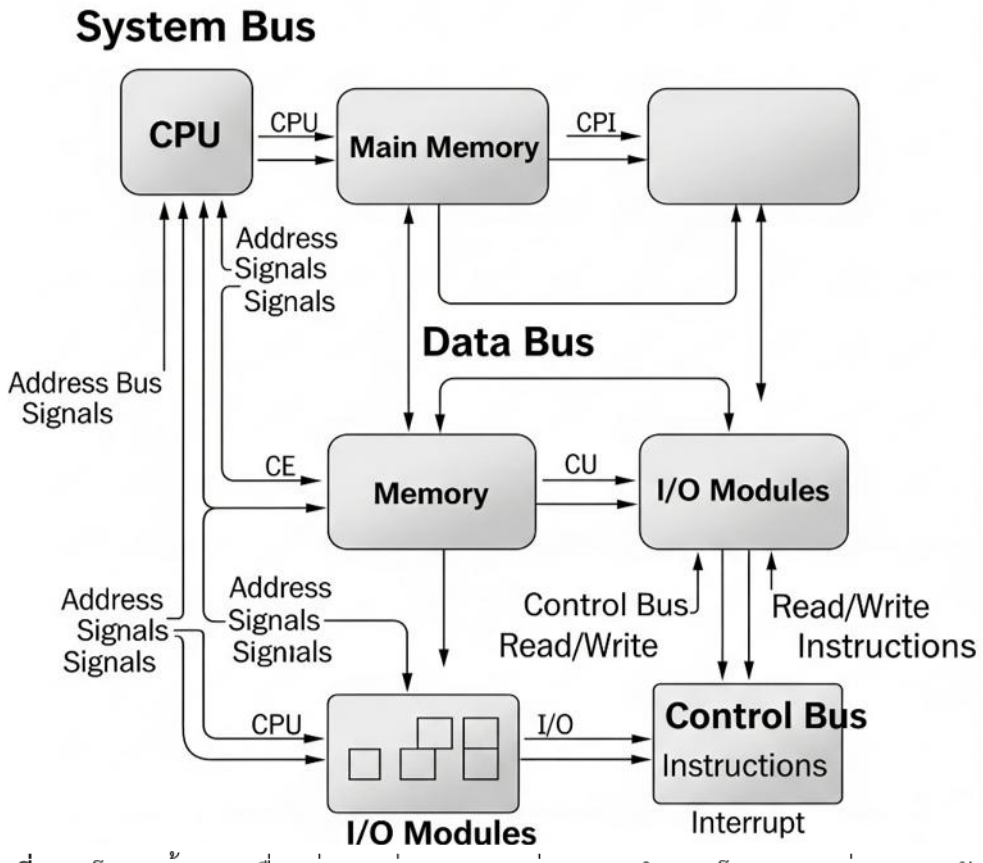
วัตถุประสงค์การเรียนรู้ (Learning Objectives)

1. อธิบายโครงสร้างการเชื่อมโยงภายในคอมพิวเตอร์ (Computer Interconnection Structure) ได้อย่างถูกต้อง
2. อธิบายโครงสร้างการเชื่อมต่อของส่วนหน่วยประมวลผลกลาง (Central Processing Unit - CPU), หน่วยความจำหลัก (Main Memory) หน่วยรับข้อมูลและหน่วยแสดงผลข้อมูล (Input /Output - I/O Modules) ได้อย่างถูกต้อง
3. อธิบายโครงสร้างการเชื่อมต่อช่องทางสื่อสารหลักบัส (Bus System) ได้แก่ระบบบัส Address Bus, Data Bus และ Control Bus ได้อย่างถูกต้อง
4. อธิบายการเชื่อมต่อสัญญาณบัสกับอุปกรณ์ ประมวลผล หน่วยความจำ รับข้อมูล และแสดงผลข้อมูล ได้อย่างถูกต้อง
5. บอกการทำงานระบบบัสของหน่วยประมวลผลกลาง (CPU Bus) ได้อย่างถูกต้อง
6. อธิบายหน้าที่การทำงานระบบบัสของหน่วยประมวลผลกลาง (CPU Bus) ของสัญญาณ ดึงคำสั่ง (Instruction Fetch), เข้าถึงข้อมูล (Data Access), ควบคุมอุปกรณ์ (Device Control), และตอบสนองการขัดจังหวะ (Interrupt Handling) ได้อย่างถูกต้อง.
7. อธิบายประเภทการทำงานระบบบัสของหน่วยประมวลผลกลาง (CPU Bus) ประเภท ชุดคำสั่ง (Instruction), ข้อมูล (Data), สัญญาณอินเทอร์รัปต์ (Interrupt Signals), แอดเดรส (Address) และปะเภทสัญญาณควบคุม (Control Signals) ได้อย่างถูกต้อง.
8. บอกหนาที่หลักของระบบบัสหน่วยความจำ (Memory Bus) ได้อย่างถูกต้อง.
9. อธิบายสัญญาณในการทำงานของบัสหน่วยความจำ (Memory Bus) ได้อย่างถูกต้อง.
10. อธิบายการทำงานระบบบัสของหน่วยรับข้อมูลและแสดงผลข้อมูล (I/O Bus System) ได้อย่างถูกต้อง.
11. อธิบายการโอนย้ายข้อมูลระบบบัส (System Bus) กับ อุปกรณ์รับและแสดงผล (I/O Module Device) ได้อย่างถูกต้อง.
12. อธิบายวิธีการโอนย้ายข้อมูลระบบบัส (System Bus) ผ่านพอร์ต (Port) และการทำงานของพอร์ต ได้อย่างถูกต้อง.
13. อธิบายบทบาทในการโอนย้ายข้อมูลของระบบบัส (System Bus) ไปยังพอร์ต (Port) ได้อย่างถูกต้อง.
14. อธิบายเทคนิคและวิธีการโอนย้ายข้อมูลระบบบัส (System Bus) ไปยังอุปกรณ์รับข้อมูลและแสดงผลข้อมูล (I/O Techniques) ได้อย่างถูกต้อง.
15. อธิบายวงจรการควบคุมระบบบัส (System Bus) ผ่านการอินเทอร์รัพท์คอนโทลเลอร์ (Interrupt Controller) ได้อย่างถูกต้อง

16. อธิบายการจัดลำดับความสำคัญของ Interrupt (Priority Handling) แบบ Nonmaskable Interrupt (NMI) และแบบ Maskable Interrupt (MI) ได้อย่างถูกต้อง
17. อธิบายการทำงานของ Hardware Interrupt และ Software Interrupt ได้อย่างถูกต้อง
18. อธิบายการทำงานและความแตกต่างระหว่าง ของ Non-Vectored Interrupt และ Vector Interrupt ได้อย่างถูกต้อง
19. อธิบายขั้นตอนการทำงานของ Vectored Interrupt เปรียบเทียบกับการทำงานจริงได้อย่างถูกต้อง
20. อธิบายการควบคุมระบบบัส (System Bus) เพื่อโอนย้ายข้อมูลกับหน่วยความจำโดยตรง (Direct Memory Access - DMA) ได้อย่างถูกต้อง
21. อธิบายการทำงานของ DMA เมื่อ CPU ต้องการอ่านหรือบันทึกข้อมูลจำนวนมากจากฮาร์ดดิสก์ (I/O Device) ได้อย่างถูกต้อง

1. โครงสร้างการเชื่อมโยงภายในคอมพิวเตอร์ (Computer Interconnection Structure)

ในระบบคอมพิวเตอร์สมัยใหม่นั้น มีส่วนประกอบหลักหลายส่วนที่ต้องทำงานร่วมกันอย่างเป็นระบบ การสื่อสารระหว่างส่วนประกอบเหล่านี้เป็นสิ่งสำคัญยิ่งเพื่อให้คอมพิวเตอร์สามารถประมวลผลข้อมูลและทำงานได้อย่างสมบูรณ์ โครงสร้างการเชื่อมโยงภายในคอมพิวเตอร์จึงหมายถึงสถาปัตยกรรมที่อธิบายถึงวิธีการที่ส่วนประกอบหลักของคอมพิวเตอร์เชื่อมต่อและสื่อสารกัน โดยมี "บัส" (Bus) เป็นหัวใจสำคัญของการเชื่อมโยง



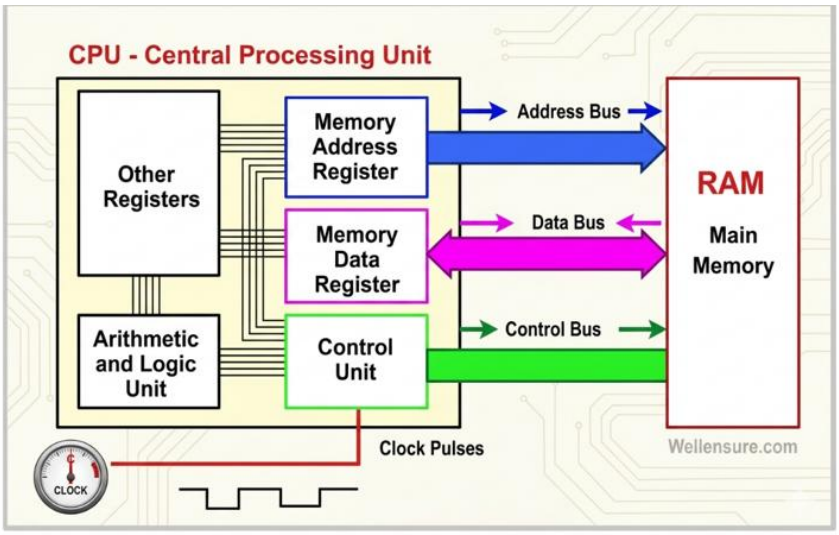
รูปที่ 5-1 โครงสร้างการเชื่อมต่อระหว่าง CPU, หน่วยความจำ และโมดูล I/O ผ่านระบบบัส

1.1 โครงสร้างและการเชื่อมต่อของส่วนประกอบหลัก: โดยทั่วไปแล้ว ส่วนประกอบหลักของระบบคอมพิวเตอร์ที่ต้องมีการเชื่อมโยงกัน ได้แก่:

- **หน่วยประมวลผลกลาง (Central Processing Unit - CPU):** ทำหน้าที่ควบคุมและประมวลผลคำสั่งต่างๆ.
- **หน่วยความจำหลัก (Main Memory):** ทำหน้าที่เก็บข้อมูลและชุดคำสั่งที่กำลังถูกใช้งาน.
- **หน่วยรับและแสดงผล (Input/Output - I/O Modules):** ทำหน้าที่เชื่อมต่อกับอุปกรณ์ภายนอกต่างๆ เช่น คีย์บอร์ด, เมาส์, จอภาพ, เครื่องพิมพ์, ฮาร์ดดิสก์, การ์ดเครือข่าย เป็นต้น.

สรุป ส่วนประกอบเหล่านี้ไม่สามารถทำงานแยกกันได้ ต้องมีการแลกเปลี่ยนข้อมูลและสัญญาณควบคุมระหว่างกันตลอดเวลา และ "บัส" ก็คือเส้นทางหรือช่องทางที่ใช้ในการสื่อสาร

2.อธิบายโครงสร้างการเชื่อมต่อช่องทางสื่อสารหลักบัส (Bus System):



รูปที่ 5-2 โครงสร้างการเชื่อมต่อระหว่าง CPU, หน่วยความจำ และโมดูล I/O ผ่านระบบบัส

บัสคือกลุ่มของสายสัญญาณที่ทำหน้าที่เป็นช่องทางร่วมในการส่งข้อมูล, แอดเดรส (ตำแหน่ง), และสัญญาณควบคุมระหว่างส่วนประกอบต่างๆ ในระบบคอมพิวเตอร์ บัสหลักที่สำคัญมี 3 ชนิด ได้แก่:

2.1 Address Bus (บัสแอดเดรส):

- **หน้าที่:** ใช้ส่งสัญญาณที่ระบุ "ตำแหน่ง" ของข้อมูลที่ CPU ต้องการอ่านหรือเขียนไปยังหน่วยความจำ หรือไปยังอุปกรณ์ I/O ที่เฉพาะเจาะจง.
- **ลักษณะ:** เป็นบัสแบบทางเดียว (Unidirectional) คือข้อมูลแอดเดรสจะถูกส่งออกจาก CPU ไปยังหน่วยความจำหรือ I/O เท่านั้น.จำนวนสายของบัสแอดเดรสจะเป็นตัวกำหนดขนาดของหน่วยความจำสูงสุดที่ระบบอ้างอิงได้ (เช่น บัส 32 บิต สามารถอ้างอิงได้ $2^{\wedge 32}$ ตำแหน่ง).

2.2 Data Bus (บัสข้อมูล):

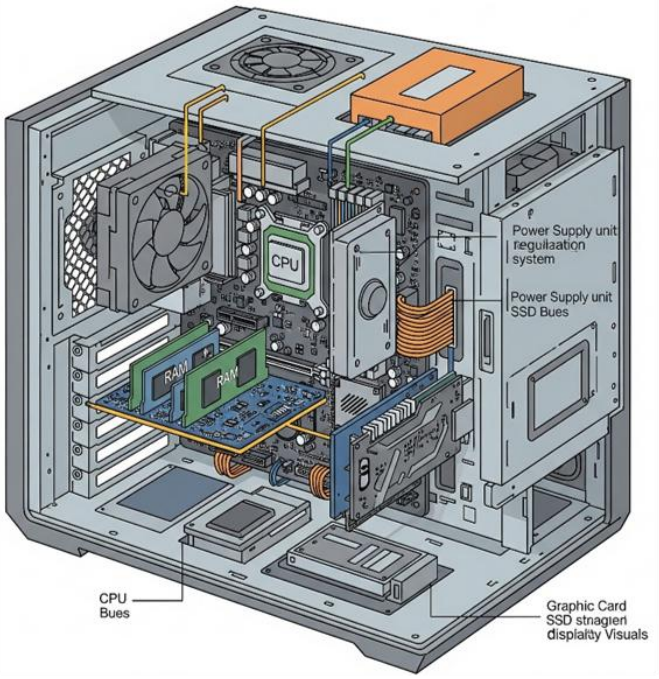
- **หน้าที่:** ส่ง "ข้อมูล" จริงๆ ที่ต้องการโอนระหว่าง CPU, หน่วยความจำ, และอุปกรณ์ I/O.
- **ลักษณะ:** เป็นบัสแบบสองทาง (Bidirectional) คือข้อมูลสามารถไหลไปได้ทั้งสองทิศทาง (จาก CPU ไป Memory/I/O หรือจาก Memory/I/O มายัง CPU). ความกว้างของบัสข้อมูล (จำนวนสาย) จะส่งผลต่อปริมาณข้อมูลที่สามารถถ่ายโอนได้ในแต่ละครั้ง (เช่น บัส 64 บิต สามารถส่งข้อมูลได้ 64 บิตพร้อมกัน).

2.3 Control Bus (บัสควบคุม):

- **หน้าที่:** ใช้ส่ง "สัญญาณควบคุม" ต่างๆ ที่จำเป็นต่อการจัดการและประสานงานการทำงานของส่วนประกอบในระบบ. สัญญาณเหล่านี้จะบอกว่ากำลังจะทำอะไร เช่น อ่านข้อมูล, เขียนข้อมูล, ร้องขอการใช้บัส, หรือตอบรับการทำงาน.
- **ลักษณะ:** เป็นบัสแบบสองทาง (Bidirectional) สัญญาณควบคุมสามารถส่งไปได้ทั้งสองทิศทางได้แก่:
  - **Memory Read/Write:** สัญญาณบอกว่ากำลังจะอ่านหรือเขียนข้อมูลไปยังหน่วยความจำ.

- **I/O Read/Write:** สัญญาณบอกว่ากำลังจะอ่านหรือเขียนข้อมูลจาก CPU ไปยังอุปกรณ์ I/O.
- **Bus Request/Grant:** สัญญาณร้องขอและอนุญาตให้ใช้บัส (เมื่อมีอุปกรณ์หลายตัวต้องการใช้บัสพร้อมกัน).
- **Interrupt Request/Acknowledge:** สัญญาณการร้องขอขัดจังหวะและการตอบรับจาก CPU.
- **Clock:** สัญญาณนาฬิกาที่ใช้ในการซิงโครไนซ์การทำงานของส่วนประกอบต่างๆ.

3.การเชื่อมต่อสัญญาณบัสกับอุปกรณ์ ประมวลผล หน่วยความจำ รับข้อมูล และแสดงผลข้อมูล :

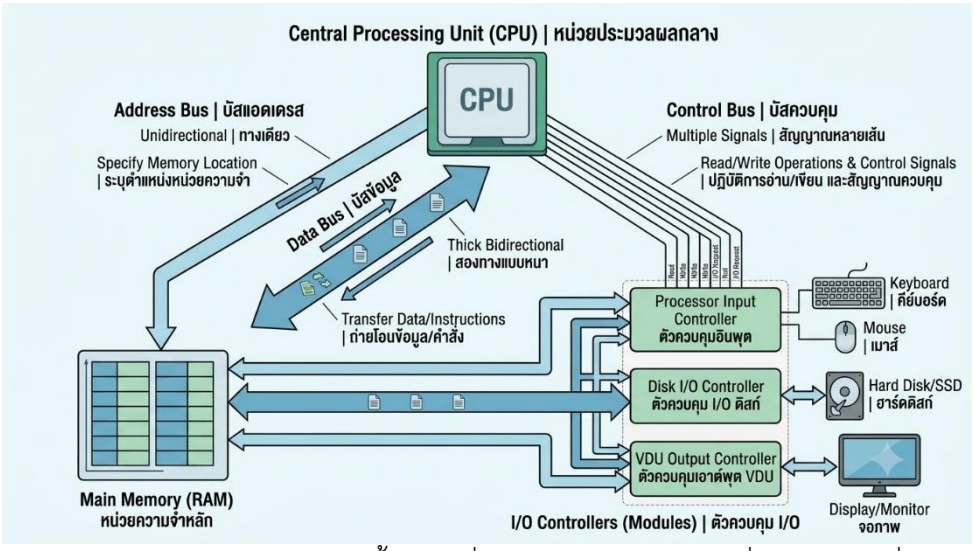


รูปที่ 5-3 การจัดวางอุปกรณ์และการเชื่อมต่อระบบบัสภายในเครื่องคอมพิวเตอร์

จากรูป CPU: จะมีเส้นสายที่แยกออกไป 3 กลุ่มใหญ่ๆ คือ **Address Bus, Data Bus,** และ **Control Bus**. ด้านหนึ่งของ CPU: จะมี **Main Memory** (หน่วยความจำหลัก) เชื่อมต่ออยู่ส่วนและจะมีพอร์ตเชื่อมต่อรับสัญญาณจาก Address Bus (เพื่อรับตำแหน่งที่ต้องการเข้าถึง), Data Bus (เพื่อรับ/ส่งข้อมูล), และ Control Bus (เพื่อรับสัญญาณ Read/Write).

3.1 ส่วน CPU: จะมี **I/O Modules** (โมดูลควบคุม I/O) เชื่อมต่ออยู่ ซึ่งแต่ละโมดูลจะทำหน้าที่เชื่อมต่อกับอุปกรณ์ I/O เฉพาะ.

- **Processor Input Controller (เช่น สำหรับ Keyboard/Mouse):** เชื่อมต่อกับบัสทั้งสามเพื่อรับ/ส่งข้อมูลจากอุปกรณ์อินพุต.
- **Disk I/O Controller (เช่น สำหรับ Hard Disk/SSD):** เชื่อมต่อกับบัสทั้งสามเพื่อจัดการการอ่าน/เขียนข้อมูลกับอุปกรณ์เก็บข้อมูล.
- **VDU Output Controller (Video Display Unit, เช่น สำหรับ Monitor/Display):** เชื่อมต่อกับบัสทั้งสามเพื่อส่งข้อมูลภาพออกไปยังหน้าจอ.
- แต่ละ Controller (ตัวควบคุม) นี้จะมีวงจรเฉพาะของตัวเองในการสื่อสารกับอุปกรณ์ I/O ที่ควบคุม และในขณะเดียวกันก็สื่อสารกับ CPU ผ่านบัส.



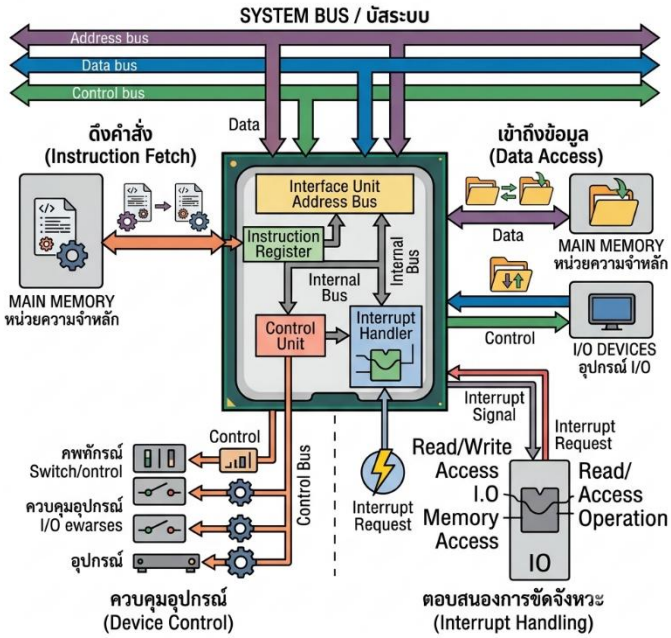
ดังนั้นเป็นเหมือน "โครงข่ายคอมพิวเตอร์" ที่มี CPU เป็นศูนย์กลาง มีสายบัสทั้งสามชนิดพาดผ่านเชื่อมต่อไปยัง Memory และ I/O Modules ต่างๆ ซึ่ง I/O Modules เหล่านี้จะมีเส้นเชื่อมต่อไปยังอุปกรณ์ I/O ภายนอกอีกทีโดยมี

-Address Bus: เป็นเส้นเดียวที่ออกจาก CPU ไปยังทุกส่วนประกอบที่สามารถอ้างอิงตำแหน่งได้ (Memory, I/O Modules).

-Data Bus: เป็นเส้นหนาที่พาดผ่านเชื่อมโยงทุกส่วนประกอบเข้าด้วยกัน เพื่อให้ข้อมูลไหลได้สองทาง

-**Control Bus:** เป็นกลุ่มของเส้นสัญญาณหลายเส้นที่เชื่อมโยงทุกส่วนประกอบ เพื่อส่งสัญญาณควบคุมต่างๆ.

4. การทำงานระบบบัสของหน่วยประมวลผลกลาง (CPU Bus): บัสของหน่วยประมวลผลกลาง หรือที่เราเรียกว่า **CPU Bus** (หรือบางครั้งเรียกว่า System Bus) คือเส้นทางหลักที่หน่วยประมวลผลกลาง (CPU) ใช้ในการสื่อสารและแลกเปลี่ยนข้อมูลกับส่วนประกอบหลักอื่นๆ เช่น หน่วยความจำ (Main Memory) และอุปกรณ์ Input/Output (I/O) ต่างๆ บัสนี้เป็นหัวใจสำคัญที่ทำให้ CPU สามารถเข้าถึงข้อมูลและสั่งการอุปกรณ์ต่างๆ ได้อย่างมีประสิทธิภาพ



รูปที่ 5-4: แผนภูมิการทำงานของระบบบัส CPU (CPU Bus System Operation Diagram)

4.1 หน้าที่หลักของ CPU Bus: คือการเป็นช่องทางที่ช่วยให้ CPU สามารถ:

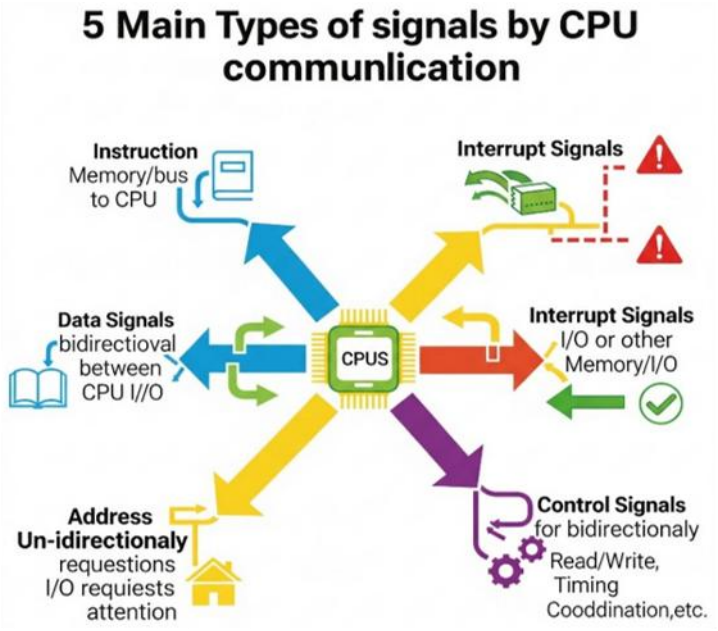
4.1.1 ดึงคำสั่ง (Instruction Fetch): ดึงชุดคำสั่งจากหน่วยความจำมาประมวลผล.

4.1.2เข้าถึงข้อมูล (Data Access): อ่านหรือเขียนข้อมูลไปยังหน่วยความจำ หรือไปยังอุปกรณ์ I/O.

4.1.3 ควบคุมอุปกรณ์ (Device Control): ส่งสัญญาณควบคุมไปยังอุปกรณ์ต่างๆ เพื่อให้ทำงานตามที่ต้องการ.

4.1.4.ตอบสนองการขัดจังหวะ (Interrupt Handling): รับและตอบสนองสัญญาณขัดจังหวะจากอุปกรณ์ I/O.

4.2 ประเภทของสัญญาณที่ CPU ใช้ในการสื่อสารผ่านบัส:



รูปที่ 5-5 ประเภทสัญญาณหลักในการสื่อสารของหน่วยประมวลผลกลาง

CPU Bus ไม่ได้มีเพียงสายสัญญาณประเภทเดียว แต่ประกอบด้วยกลุ่มของสายสัญญาณหลายประเภทที่ทำหน้าที่แตกต่างกัน โดยแบ่งออกเป็น 5 ประเภทหลักๆ ดังนี้:

4.2.1 ชุดคำสั่ง (Instruction) :

หน้าที่: นี่คือ "สิ่งที่ CPU กำลังจะทำ" หรือ "คำสั่งที่บอกให้ CPU ทำงาน".

รายละเอียด: เมื่อ CPU ต้องการประมวลผลงานใดๆ จะต้องดึงชุดคำสั่ง (Instruction) จากหน่วยความจำมาที่ตัวมันเองก่อน ชุดคำสั่งเหล่านี้จะถูกส่งผ่านบัสข้อมูล (Data Bus) และ CPU จะตีความว่าคำสั่งนั้นคืออะไร เช่น คำสั่งบวกเลข, คำสั่งย้ายข้อมูล, หรือคำสั่งกระโดดไปทำส่วนอื่นของโปรแกรม.

การเดินทาง: ชุดคำสั่งจะถูกส่งจากหน่วยความจำมายัง CPU ผ่าน Data Bus.

4.2.2 ข้อมูล (Data):

หน้าที่: นี่คือ "ข้อมูลจริง" ที่ CPU ใช้ในการประมวลผล หรือข้อมูลที่ต้องการโอนย้ายไปมาระหว่าง CPU กับหน่วยความจำ หรืออุปกรณ์ I/O.

- **รายละเอียด:** ข้อมูลอาจเป็นตัวเลข, ตัวอักษร, รูปภาพ, หรืออะไรก็ตามที่คอมพิวเตอร์สามารถจัดการได้ เมื่อ CPU ต้องการอ่านข้อมูลจากหน่วยความจำ, จากคีย์บอร์ด, หรือต้องการส่งข้อมูลผลลัพธ์ไปเก็บในหน่วยความจำ หรือส่งออกไปยังจอภาพ ข้อมูลเหล่านั้นจะเดินทางผ่านบัสข้อมูล (Data Bus).
- **การเดินทาง:** ข้อมูลสามารถเดินทางได้สองทิศทาง (จาก CPU ไป Memory/I/O หรือจาก Memory/I/O มายัง CPU) ผ่าน Data Bus.

4.2.3 สัญญาณอินเทอร์รัปต์ (Interrupt Signals) :

- **หน้าที่:** เป็น "สัญญาณขอความสนใจ" ที่บ่งบอกว่ามีอุปกรณ์ใดต้องการติดต่อกับ CPU หรือต้องการให้ CPU หยุดการทำงานชั่วคราวเพื่อไปจัดการงานเร่งด่วน.
- **รายละเอียด:** ปกติแล้ว CPU จะทำงานตามลำดับคำสั่งที่ได้รับมา แต่ในบางสถานการณ์ เช่น เมื่อผู้ใช้กดแป้นพิมพ์ หรือเมื่อมีข้อผิดพลาดเกิดขึ้น อุปกรณ์ I/O เหล่านี้จะส่งสัญญาณ Interrupt ไปยัง CPU เพื่อแจ้งว่ามีเหตุการณ์สำคัญเกิดขึ้นและต้องการให้ CPU จัดการ. CPU จะต้องหยุดงานที่กำลังทำอยู่ชั่วคราวเพื่อไปบริการ Interrupt นั้นก่อน.
- **การเดินทาง:** สัญญาณ Interrupt ส่งจาก I/O Module มายัง CPU ผ่านสายสัญญาณ Control Bus.

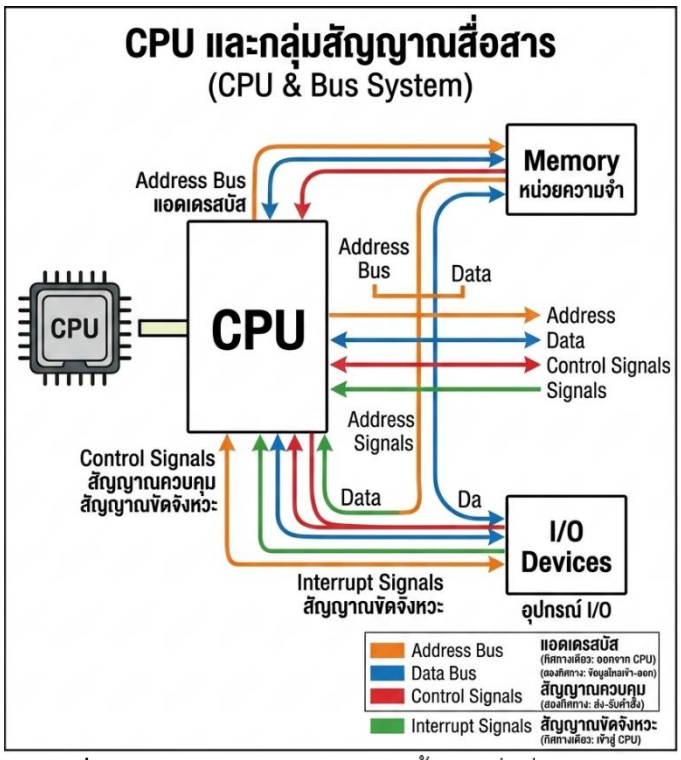
4.2.4 แอดเดรส (Address) :

- **หน้าที่:** เป็น "สัญญาณบอกตำแหน่ง" ที่ CPU ต้องการติดต่อด้วย ไม่ว่าจะเป็นตำแหน่งของข้อมูลในหน่วยความจำ หรือตำแหน่งของพอร์ตของอุปกรณ์ I/O.
- **รายละเอียด:** ทุกๆ ไบต์ของข้อมูลในหน่วยความจำจะมีแอดเดรสเฉพาะตัว และทุกๆ พอร์ตของอุปกรณ์ I/O ก็จะมีแอดเดรสเฉพาะตัวเช่นกัน เมื่อ CPU ต้องการอ่านหรือเขียนข้อมูลที่ตำแหน่งใดๆ หรือต้องการสื่อสารกับอุปกรณ์ I/O ตัวใด CPU จะส่งสัญญาณแอดเดรสที่ถูกต้องออกไปทางบัสแอดเดรส (Address Bus) เพื่อระบุตำแหน่งเป้าหมาย.
- **การเดินทาง:** สัญญาณแอดเดรสจะถูกส่งออกจาก CPU ไปยังหน่วยความจำหรือ I/O Module เท่านั้น (เป็นแบบทางเดียว) ผ่าน Address Bus.

4.2.5 สัญญาณควบคุม (Control Signals) :

- **หน้าที่:** เป็น "สัญญาณสั่งการ" หรือ "สัญญาณกำกับ" ที่ CPU ส่งออกไปควบคุมอุปกรณ์ต่างๆ ให้ทำงานตามต้องการ หรือเป็นสัญญาณที่อุปกรณ์ส่งกลับมาเพื่อยืนยันสถานะ.
- **รายละเอียด:** สัญญาณเหล่านี้มีความหลากหลายมาก เช่น:
  - **Memory Read:** CPU บอกว่า "ฉันต้องการอ่านข้อมูลจากหน่วยความจำนะ".
  - **Memory Write:** CPU บอกว่า "ฉันต้องการเขียนข้อมูลลงในหน่วยความจำนะ".
  - **I/O Read:** CPU บอกว่า "ฉันต้องการอ่านข้อมูลจากอุปกรณ์ I/O นะ".
  - **I/O Write:** CPU บอกว่า "ฉันต้องการเขียนข้อมูลไปยังอุปกรณ์ I/O นะ".
  - **Bus Request:** อุปกรณ์อื่นร้องขอใช้บัส และ **Bus Grant:** CPU อนุญาตให้อุปกรณ์ใช้บัส.
  - **Clock:** สัญญาณนาฬิกาที่ใช้ในการซิงโครไนซ์การทำงานของส่วนประกอบทั้งหมด.

- **การเดินทาง:** สัญญาณควบคุมสามารถเดินทางได้ทั้งสองทิศทาง (จาก CPU ไปอุปกรณ์ หรือจากอุปกรณ์มายัง CPU) ผ่าน Control Bus.



รูปที่ 5-6 การจัดสรรสายสัญญาณข้อมูล ที่อยู่ และการควบคุมระหว่าง CPU กับหน่วยความจำและอุปกรณ์ I/O

<u>คำอธิบายลักษณะของรูปที่</u> 5-6:

***ตรงกลางของรูป: "CPU"** จะมีเส้นสายหลายเส้นพุ่งออกมาและ/หรือพุ่งเข้าไป เพื่อเชื่อมต่อไปยังส่วนอื่นๆ ของระบบ.

***เส้นที่พุ่งออกจาก CPU (เป็นหลัก):**

"Address Bus" และมีลูกศรชี้ออกจาก CPU ไปทางด้านนอก (อาจจะชี้ไปยังกล่อง "Memory" หรือ "I/O Modules"

"Control Signals" และมีลูกศรชี้ออกและเข้า CPU (เป็น bidirectional) เพื่อแสดงการส่งสัญญาณควบคุมต่างๆ.

***เส้นที่พุ่งเข้าและออกจาก CPU:**

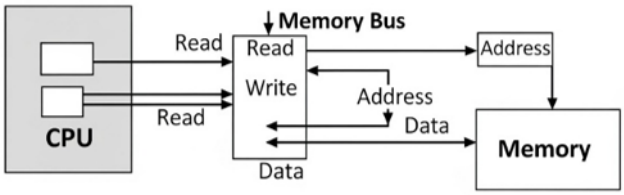
"Data Bus" และมีลูกศรทั้งชี้เข้าและออก CPU (เป็น bidirectional) เพื่อแสดงการไหลของข้อมูล.

***เส้นที่พุ่งเข้า CPU (เป็นหลัก):**

- **Interrupt Signals:** จะเป็นกลุ่มของเส้นที่เขียนกำกับว่า "Interrupt Signals" และมีลูกศรชี้เข้า CPU เพื่อแสดงว่าสัญญาณเหล่านี้มาจากอุปกรณ์ภายนอก.

*สรุปเพราะฉะนั้นภาพนี้จะเน้นให้เห็นถึง **"อินเทอร์เฟซ"** ระหว่าง CPU กับโลกภายนอกผ่านกลุ่มของสายสัญญาณที่เรียกว่าบัส ซึ่งแต่ละกลุ่มมีบทบาทเฉพาะเจาะจงในการสื่อสาร*

5.ระบบบัสของหน่วยความจำ (Memory Bus)



รูปที่ 5-7 ขั้นตอนการปฏิบัติการอ่าน/เขียนข้อมูลระหว่าง CPU และหน่วยความจำ

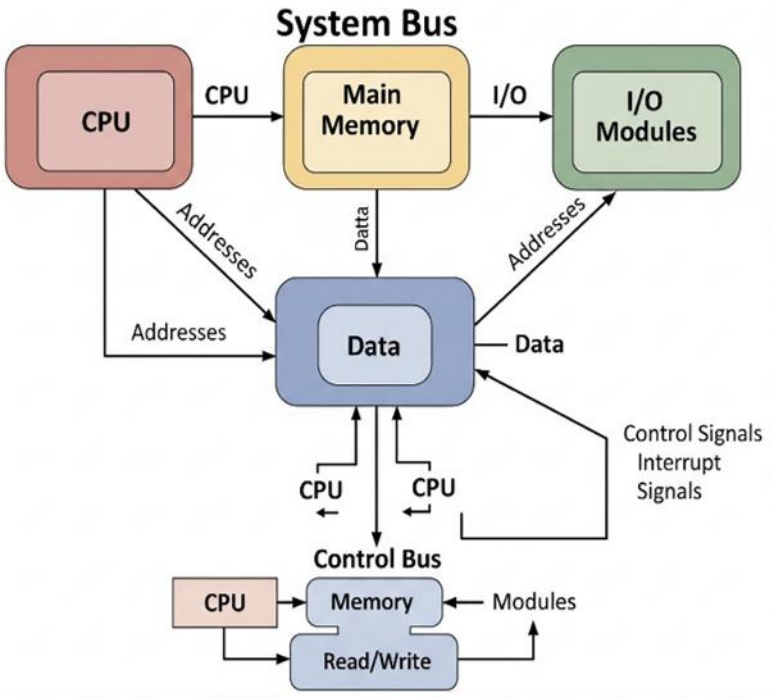
<u>บัสของหน่วยความจำ</u> หรือ **Memory Bus** คือกลุ่มของสายสัญญาณที่ทำหน้าที่เป็นช่องทางเฉพาะสำหรับการสื่อสารและแลกเปลี่ยนข้อมูลระหว่างหน่วยประมวลผลกลาง (CPU) กับหน่วยความจำหลัก (Main Memory) หน่วยความจำหลักเป็นส่วนสำคัญที่เก็บทั้งชุดคำสั่งที่ CPU กำลังประมวลผลอยู่และข้อมูลที่ใช้งานในปัจจุบัน ดังนั้นการสื่อสารที่มีประสิทธิภาพผ่าน Memory Bus จึงเป็นปัจจัยสำคัญต่อความเร็วในการทำงานโดยรวมของคอมพิวเตอร์

5.1 หน้าที่หลักของบัสหน่วยความจำ Memory Bus: มีหน้าที่หลักในการดังนี้:

1. **ถ่ายโอนคำสั่ง:** ส่งชุดคำสั่งจากหน่วยความจำไปยัง CPU เพื่อให้ CPU ดึงไปประมวลผล.
2. **ถ่ายโอนข้อมูล:** ส่งข้อมูลจากหน่วยความจำไปยัง CPU (เมื่อ CPU ต้องการอ่านข้อมูล) หรือจาก CPU ไปยังหน่วยความจำ (เมื่อ CPU ต้องการเขียนข้อมูล).
3. **ระบุตำแหน่ง:** CPU ใช้บัสแอดเดรสบน Memory Bus ในการระบุตำแหน่งที่แน่นอนในหน่วยความจำที่ต้องการอ่านหรือเขียนข้อมูล.
4. **ควบคุมการเข้าถึง:** สัญญาณควบคุมบน Memory Bus จะกำหนดว่ากำลังจะทำอะไรกับหน่วยความจำ (อ่านหรือเขียน).

5.2 สัญญาณในการทำงานของบัสหน่วยความจำ:

Memory Bus ประกอบด้วยส่วนประกอบของบัสหลักทั้งสามชนิด (Address Bus, Data Bus, Control Bus) ที่ปรับให้เข้ากับการสื่อสารกับหน่วยความจำโดยเฉพาะมีสัญญาณสำคัญดังนี้:



รูปที่ 5-8 การเชื่อมต่อและการกระจายสัญญาณของระบบบัสระหว่าง CPU, หน่วยความจำ และโมดูล I/O

5.2.1 สัญญาณสำหรับการอ่านข้อมูล (Read):

- **หน้าที่:** เป็นสัญญาณควบคุมที่ CPU ส่งออกไปยังหน่วยความจำ เพื่อ "สั่ง" ให้หน่วยความจำส่งข้อมูลที่อยู่ในตำแหน่งแอดเดรสที่ระบุออกมา.
- **รายละเอียด:** เมื่อ CPU ต้องการข้อมูลจากหน่วยความจำ มันจะส่งสัญญาณแอดเดรสผ่าน Address Bus และพร้อมกันนั้นก็ส่งสัญญาณ Read (ที่เป็นส่วนหนึ่งของ Control Bus) ไปยังหน่วยความจำ หน่วยความจำที่รับสัญญาณ Read และแอดเดรสที่ถูกต้องแล้ว จะนำข้อมูลที่อยู่ในตำแหน่งนั้นๆ ออกมาวางบน Data Bus เพื่อให้ CPU ดึงไปใช้.
- **การทำงาน:** สัญญาณนี้จะทำงานเมื่อ CPU ต้องการ **"อ่าน"** ข้อมูลจาก Memory.

5.2.2 สัญญาณสำหรับการเขียนข้อมูล (Write):

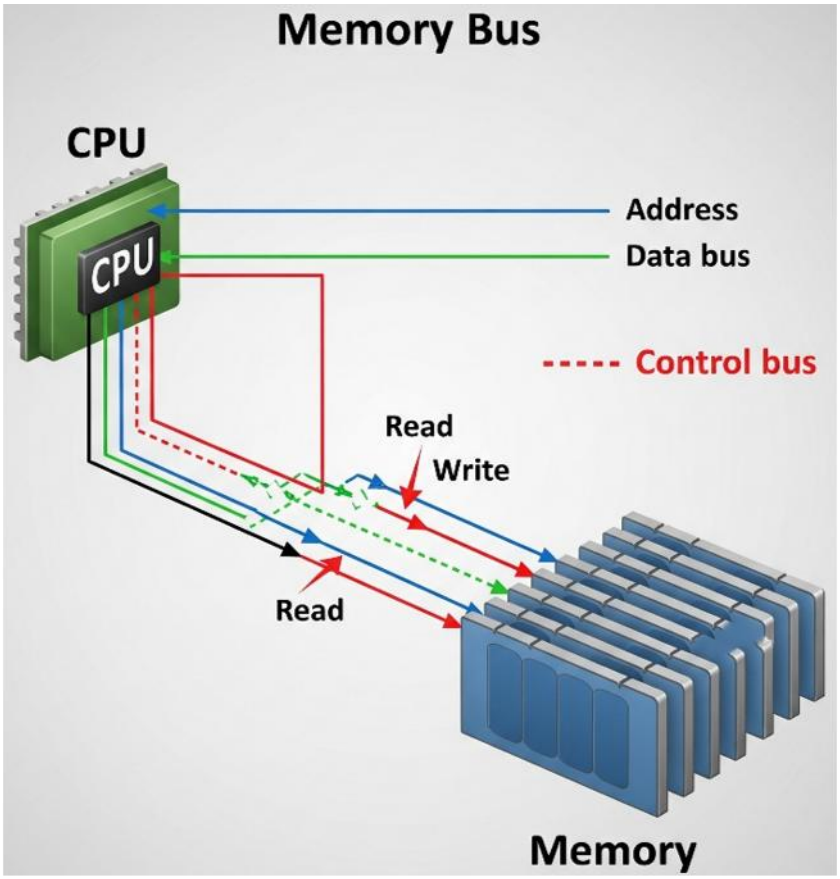
- **หน้าที่:** เป็นสัญญาณควบคุมที่ CPU ส่งออกไปยังหน่วยความจำเพื่อ "สั่ง" ให้หน่วยความจำบันทึกข้อมูลที่ CPU ส่งมาลงในตำแหน่งแอดเดรสที่ระบุ.
- **รายละเอียด:** เมื่อ CPU ต้องการเขียนข้อมูลลงในหน่วยความจำ มันจะส่งสัญญาณแอดเดรสผ่าน Address Bus, ส่งข้อมูลที่ต้องการเขียนผ่าน Data Bus, และพร้อมกันนั้นก็ส่งสัญญาณ Write (ที่เป็นส่วนหนึ่งของ Control Bus) ไปยังหน่วยความจำ หน่วยความจำที่รับสัญญาณ Write, แอดเดรส, และข้อมูลแล้ว จะบันทึกข้อมูลนั้นลงในตำแหน่งที่ระบุ.
- **การทำงาน:** สัญญาณนี้จะทำงานเมื่อ CPU ต้องการ **"เขียน"** ข้อมูลลงใน Memory.

5.2.3 สัญญาณตำแหน่งที่ใช้อ้างอิงในหน่วยความจำ (Address):

- **หน้าที่:** ใช้ระบุ "ตำแหน่ง" ของเซลล์หน่วยความจำที่ CPU ต้องการเข้าถึง.
- **รายละเอียด:** หน่วยความจำหลักถูกจัดเรียงเป็นช่องๆ เหมือนกล่องจดหมาย และแต่ละช่องมี "หมายเลข" ประจำตัว หรือที่เรียกว่าแอดเดรส (Address) เมื่อ CPU ต้องการเข้าถึงข้อมูลในตำแหน่งใด ก็จะส่งหมายเลขแอดเดรสของตำแหน่งนั้นออกไปทาง Address Bus เพื่อให้หน่วยความจำทราบว่าต้องเข้าถึงข้อมูลที่จุดใด.
- **การทำงาน:** แอดเดรสถูกส่งจาก CPU ไปยัง Memory (เป็นแบบทางเดียว) ผ่าน Address Bus.

5.2.4 สัญญาณข้อมูลที่ใช้ในการโอนย้ายระหว่าง CPU กับหน่วยความจำ (Data):

- **หน้าที่:** เป็นช่องทางสำหรับ "ข้อมูลจริง" ที่ถูกโอนย้ายระหว่าง CPU และหน่วยความจำ.
- **รายละเอียด:** ไม่ว่าจะเป็นชุดคำสั่งที่ CPU ต้องการดึงมาประมวลผล หรือข้อมูลที่ CPU ประมวลผลเสร็จแล้วและต้องการส่งกลับไปเก็บในหน่วยความจำ หรือข้อมูลที่ CPU ต้องการอ่านจากหน่วยความจำเพื่อนำไปใช้งาน ข้อมูลเหล่านี้จะเดินทางผ่าน Data Bus.
- **การทำงาน:** ข้อมูลสามารถเดินทางได้สองทิศทาง (จาก CPU ไป Memory หรือจาก Memory มายัง CPU) ผ่าน Data Bus.



รูปที่ 5-9 การแยกประเภทสายสัญญาณบนบัสหน่วยความจำสำหรับการปฏิบัติการ Read และ Write

จากรูปที่ 5-9 Module "CPU" เชื่อมต่อกับระบบบัส และมีการเชื่อมต่อไปยัง **"Main Memory"** หรือ **"RAM"** (หน่วยความจำหลัก).ซึ่งระหว่าง CPU และ Main Memory: จะมีกลุ่มของเส้นสายที่เชื่อมโยงกัน 3 กลุ่มคือ

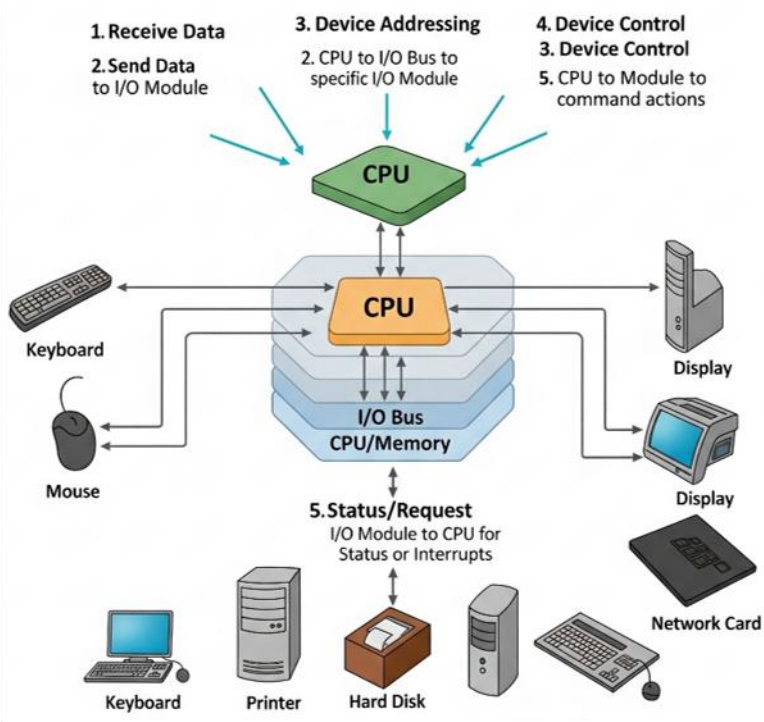
- **Address Bus:** เป็นกลุ่มของเส้นสายที่มีลูกศรชี้ **จาก CPU ไปยัง Main Memory เท่านั้น** (แสดงการส่งแอดเดรส).
- **Data Bus:** เป็นกลุ่มของเส้นสายที่มีลูกศร **ชี้เข้าและออกระหว่าง CPU และ Main Memory ทั้งสองทิศทาง** (แสดงการรับส่งข้อมูล).
- **Control Bus (ส่วนที่เกี่ยวข้องกับ Memory):** จะมีเส้นแยกออกมาจากกลุ่ม Control Bus ของ CPU ไปยัง Main Memory โดยเฉพาะ โดยจะมีสองเส้นหลักๆ ที่เขียนกำกับว่า **"Read"** และ **"Write"** พร้อมลูกศรชี้ **จาก CPU ไปยัง Main Memory เท่านั้น** (แสดงสัญญาณสั่งการ).

- **สัญญาณภายในหน่วยความจำหลัก (Internal Main Memory):** อาจจะมีการแสดงให้เห็นว่า Memory ถูกแบ่งเป็นช่องๆ และแต่ละช่องมีแอดเดรสกำกับอยู่ เพื่อแสดงให้เห็นว่า Address Bus ทำงานอย่างไรในการเลือกตำแหน่งข้อมูล.

6. ระบบบัสของหน่วยรับข้อมูลและแสดงผลข้อมูล (I/O Bus System)

บัสของหน่วยรับและแสดงผล หรือที่เรียกว่า **I/O Bus** คือกลุ่มของสายสัญญาณที่ทำหน้าที่เป็นช่องทางหลักสำหรับการสื่อสารและแลกเปลี่ยนข้อมูลระหว่างหน่วยประมวลผลกลาง (CPU) กับอุปกรณ์รับและแสดงผล (Input/Output Devices) ต่างๆ เช่น คีย์บอร์ด, เมาส์, จอภาพ, เครื่องพิมพ์, ฮาร์ดดิสก์, การ์ดเครือข่าย, หรืออุปกรณ์ต่อพ่วงอื่นๆ. แม้ว่าอุปกรณ์ I/O จะมีความหลากหลายและมีลักษณะการทำงานที่แตกต่างกันไป แต่พวกมันจำเป็นต้องมีช่องทางมาตรฐานในการเชื่อมต่อกับ CPU และหน่วยความจำ และ I/O Bus ก็ทำหน้าที่นั้น

6.1 หน้าที่หลักของ I/O Bus: มีหน้าที่หลักในการ:



รูปที่ 5-10 ลำดับขั้นตอนการประสานงานของ CPU ร่วมกับโมดูล I/O เพื่อจัดการอุปกรณ์ต่อพ่วง

6.1.1 รับข้อมูลเข้า: ถ่ายโอนข้อมูลจากอุปกรณ์รับข้อมูล (เช่น คีย์บอร์ด, เมาส์) เข้าสู่ระบบ (CPU หรือ Memory).

6.1.2 ส่งข้อมูลออก: ถ่ายโอนข้อมูลจากระบบ (CPU หรือ Memory) ไปยังอุปกรณ์แสดงผล เช่น จอภาพ , เครื่องพิมพ์.

6.1.3 ระบุอุปกรณ์: CPU ใช้บัสแอดเดรสบน I/O Bus ในการระบุต้องการสื่อสารกับอุปกรณ์ I/O ตัวใด.

6.1.4 ควบคุมอุปกรณ์: ส่งสัญญาณควบคุมไปยังอุปกรณ์ I/O เพื่อสั่งให้ทำงานบางอย่าง เช่น สั่งให้เครื่องพิมพ์เริ่มพิมพ์

6.1.5 แจ้งสถานะ/ร้องขอ: อุปกรณ์ I/O ส่งสัญญาณกลับมายัง CPU เพื่อแจ้งสถานะ หรือร้องขอการบริการ (เช่น ส่ง Interrupt).

6.2 สัญญาณในการทำงานของบัส I/O:

I/O Bus ก็เช่นเดียวกับ Memory Bus คือประกอบด้วยส่วนประกอบของบัสหลักทั้งสามชนิด (Address Bus, Data Bus, Control Bus) ให้เข้ากับการสื่อสารกับอุปกรณ์ I/O โดยมีสัญญาณสำคัญดังนี้:

6.2.1 Read (สัญญาณสำหรับการอ่านข้อมูลจากอุปกรณ์ I/O):

- **หน้าที่:** เป็นสัญญาณควบคุมที่ CPU ส่งออกไปยังโมดูลควบคุม I/O (I/O Module) ของอุปกรณ์นั้นๆ เพื่อ "สั่ง" ให้อุปกรณ์ส่งข้อมูลที่พร้อมอยู่บนบัฟเฟอร์ (Buffer) ของ I/O Module ออกมา.
- **รายละเอียด:** เมื่อ CPU ต้องการอ่านข้อมูลจากอุปกรณ์ I/O เช่น ข้อมูลการกดแป้นพิมพ์จากคีย์บอร์ดมันจะส่งสัญญาณแอดเดรสของพอร์ตคีย์บอร์ดผ่าน Address Bus และพร้อมกันนั้นก็ส่งสัญญาณ Read (ที่เป็นส่วนหนึ่งของ Control Bus) ไปยัง I/O Module ของคีย์บอร์ด I/O Module จะนำข้อมูลที่รับมาจากคีย์บอร์ด (อาจจะเก็บอยู่ในบัฟเฟอร์ชั่วคราว) ออกมาวางบน Data Bus เพื่อให้ CPU ดึงไปใช้
- **การทำงาน:** สัญญาณนี้จะทำงานเมื่อ CPU ต้องการ **"อ่าน"** ข้อมูลจากอุปกรณ์ I/O.

6.2.2 Write (สัญญาณสำหรับการเขียนข้อมูลไปยังอุปกรณ์ I/O):

- **หน้าที่:** เป็นสัญญาณควบคุมที่ CPU ส่งออกไปยัง I/O Module ของอุปกรณ์นั้นๆ เพื่อ "สั่ง" ให้บันทึกข้อมูลที่ CPU ส่งมาลงในบัฟเฟอร์ของ I/O Module เพื่อเตรียมส่งออกไปยังอุปกรณ์ I/O.
- **รายละเอียด:** เมื่อ CPU ต้องการเขียนข้อมูลไปยังอุปกรณ์ I/O เช่น ข้อมูลภาพที่จะแสดงบนจอภาพ หรือข้อมูลที่จะพิมพ์ออกทางเครื่องพิมพ์ มันจะส่งสัญญาณแอดเดรสของพอร์ตอุปกรณ์นั้นผ่าน Address Bus, ส่งข้อมูลที่ต้องการเขียนผ่าน Data Bus, และพร้อมกันนั้นก็ส่งสัญญาณ Write (ที่เป็นส่วนหนึ่งของ Control Bus) ไปยัง I/O Module ของอุปกรณ์ I/O Module รับข้อมูลไปเก็บไว้และจัดการส่งออกไปยังอุปกรณ์ I/O จริงๆ.
- **การทำงาน:** สัญญาณนี้จะทำงานเมื่อ CPU ต้องการ **"เขียน"** ข้อมูลลงในอุปกรณ์ I/O.

6.2.3 Address (ตำแหน่งที่อยู่ของอุปกรณ์ I/O ที่ใช้อ้างอิงในการติดต่อ):

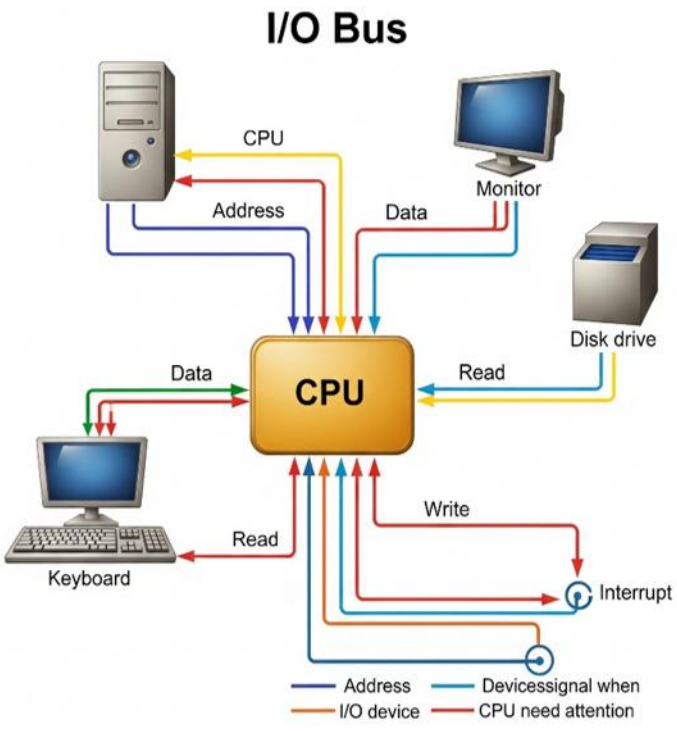
- **หน้าที่:** ใช้ระบุ "ตำแหน่ง" หรือ "พอร์ต" ของอุปกรณ์ I/O ที่ CPU ต้องการสื่อสารด้วย.
- **รายละเอียด:** อุปกรณ์ I/O แต่ละตัว หรือแม้กระทั่งฟังก์ชันย่อยๆ ภายใน I/O Module หนึ่งๆ อาจจะมีแอดเดรสเฉพาะตัว หรือพอร์ตแอดเดรสที่ CPU ใช้อ้างอิง เมื่อ CPU ต้องการสื่อสารกับอุปกรณ์ I/O ตัวใด CPU จะส่งแอดเดรสของอุปกรณ์นั้นออกไปทาง Address Bus เพื่อให้ I/O Module ที่เกี่ยวข้องทราบว่าข้อมูลหรือคำสั่งนี้มีไว้สำหรับตน.

- **การทำงาน:** แอดเดรสถูกส่งจาก CPU ไปยัง I/O Module (เป็นแบบทางเดียว) ผ่าน Address Bus.

6.2.4 Data (ข้อมูลที่ใช้รับส่งระหว่าง CPU และ I/O):

- **หน้าที่:** เป็นช่องทางสำหรับ "ข้อมูลจริง" ที่ถูกโอนย้ายระหว่าง CPU กับ I/O Module.
- **รายละเอียด:** ไม่ว่าจะเป็นข้อมูลที่อ่านจากอุปกรณ์อินพุต หรือข้อมูลที่จะเขียนไปยังอุปกรณ์เอาต์พุต ข้อมูลเหล่านี้จะเดินทางผ่าน Data Bus.
- **การทำงาน:** ข้อมูลสามารถเดินทางได้สองทิศทาง (จาก CPU ไป I/O หรือจาก I/O มายัง CPU) ผ่าน Data Bus.

6.2.5 .Interrupt Signals (สัญญาณจากอุปกรณ์ I/O ที่ต้องการติดต่อกับ CPU):



รูปที่ 5-11 ผังการเชื่อมต่อสัญญาณระหว่าง CPU และอุปกรณ์ต่อพ่วงต่าง ๆ ผ่านระบบบัส I/O

หน้าที่: เป็น "สัญญาณขอความสนใจ" ที่อุปกรณ์ I/O ส่งไปยัง CPU เพื่อแจ้งว่างานบางอย่างเสร็จสมบูรณ์แล้ว (เช่น การรับข้อมูลจากคีย์บอร์ด) หรือเกิดเหตุการณ์สำคัญที่ต้องการให้ CPU จัดการทันที (เช่น มีข้อผิดพลาดเกิดขึ้น).

รายละเอียด: โดยปกติ CPU จะไม่สามารถตรวจสอบสถานะของอุปกรณ์ I/O ได้ตลอดเวลา สัญญาณ Interrupt ช่วยให้อุปกรณ์ I/O สามารถ "ขัดจังหวะ" การทำงานปกติของ CPU เพื่อแจ้งให้ CPU ทราบถึงความจำเป็นในการดำเนินการบางอย่าง. CPU จะหยุดงานที่กำลังทำอยู่ชั่วคราว เพื่อไปบริการ Interrupt นั้นก่อน.

การทำงาน: สัญญาณ Interrupt ถูกส่งจาก I/O Module มายัง CPU (เป็นแบบทางเดียว) ซึ่งเป็นส่วนหนึ่งของ Control Bus.

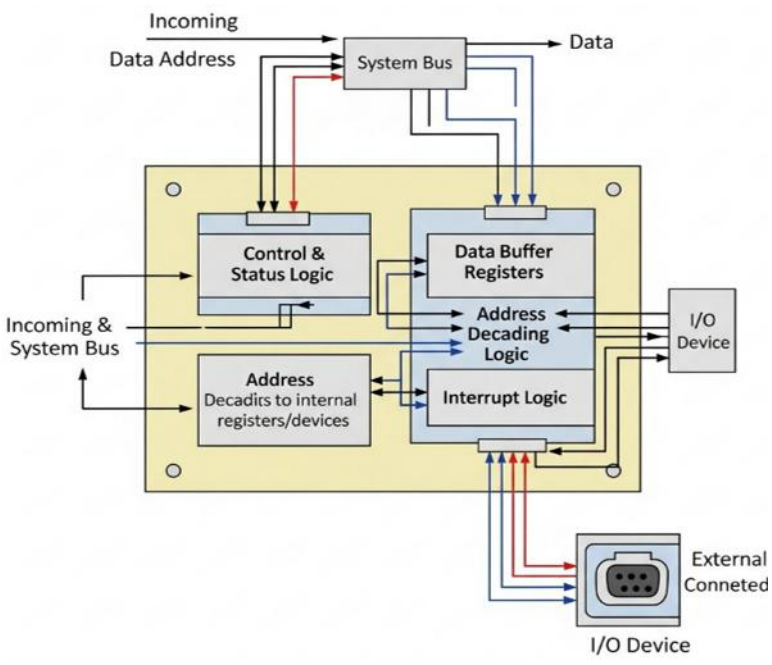
คำอธิบายลักษณะของรูปที่ 5-11

บัสของหน่วยรับและแสดงผล (I/O Bus)" นี้ จะมักมีลักษณะดังนี้

- **ตรงกลางมี "CPU"** ส่วน **"Main Memory"** เชื่อมด้วย System Bus).ต่างๆ
- **รอบๆตัว CPU จะมี "I/O Modules"** หลายๆ ตัว เช่น "Keyboard Controller", "Disk Controller", "Video Controller"

- **โดยระหว่าง CPU/Memory และ I/O Modules:** จะมีกลุ่มของเส้นสายที่เชื่อมโยงกัน:
    - **Address Bus:** เป็นกลุ่มของเส้นสายที่มีลูกศรชี้ **จาก CPU ไปยัง I/O Modules ทุกตัว** (เป็นทางเดียว). แต่ละ I/O Module จะมีวงจรตรวจสอบแอดเดรสเพื่อดูว่าสัญญาณแอดเดรสที่มานั้นตรงกับพอร์ตของตนหรือไม่.
    - **Data Bus:** เป็นกลุ่มของเส้นสายที่มีลูกศร **ชี้เข้าและออกระหว่าง CPU/Memory และ I/O Modules ทุกตัว** (เป็นสองทาง).
    - **Control Bus (ส่วนที่เกี่ยวข้องกับ I/O):** จะมีเส้นแยกออกมาจากกลุ่ม Control Bus ของ CPU ไปยัง I/O Modules โดยเฉพาะ โดยจะมีเส้นหลักๆ เช่น **"I/O Read"** และ **"I/O Write"** พร้อมลูกศรชี้ **จาก CPU ไปยัง I/O Modules เท่านั้น**. นอกจากนี้ยังจะมีเส้น **"Interrupt Request"** ที่มีลูกศรชี้ **จาก I/O Modules ไปยัง CPU** (แสดงว่า I/O ส่งสัญญาณขอความสนใจ).

7. การโอนย้ายข้อมูลระบบบัส (System Bus) กับอุปกรณ์รับและแสดงผล (I/O Module Device)



รูปที่ 5-12 แผนผังโครงสร้างการทำงานภายในของโมดูล I/O และส่วนประสานงานระหว่างระบบบัสกับอุปกรณ์ภายนอก

ในระบบคอมพิวเตอร์ อุปกรณ์รับและแสดงผล (Input/Output Devices) ต่างๆ เช่น คีย์บอร์ด, เมาส์, เครื่องพิมพ์, ฮาร์ดดิสก์, หรือจอภาพ จะมีคุณสมบัติทางไฟฟ้าและลักษณะการทำงานที่แตกต่างจากหน่วยประมวลผลกลาง (CPU) และหน่วยความจำหลัก (Main Memory) อย่างมาก ดังนั้น เพื่อให้ CPU สามารถสื่อสารกับอุปกรณ์ I/O เหล่านี้ได้อย่างมีประสิทธิภาพและปลอดภัย จึงจำเป็นต้องมี "I/O Module" หรือที่เรียกว่า "I/O Controller" ทำหน้าที่เป็นตัวกลางในการประสานงานการสื่อสาร

สรุป I/O Module (โมดูลควบคุม I/O) : คือวงจรฮาร์ดแวร์ที่ทำหน้าที่เป็น "อินเทอร์เฟซ" (Interface) หรือ "ตัวเชื่อมต่อ" ระหว่างระบบบัสหลักของคอมพิวเตอร์ (CPU, Memory) กับอุปกรณ์ I/O ภายนอก โดยแต่ละ I/O Module มักจะออกแบบมาเพื่อรองรับอุปกรณ์ I/O ประเภทใดประเภทหนึ่ง หรือกลุ่มของอุปกรณ์ I/O ที่มีลักษณะคล้ายกัน (เช่น Serial Port Controller, Parallel Port Controller, Disk Controller)

การทำงานของวงจรสื่อสารข้อมูลภายใน I/O Module: ไม่ได้เป็นเพียงสายเชื่อมต่อธรรมดา แต่ภายในประกอบด้วยวงจรที่ซับซ้อนเพื่อจัดการกับการสื่อสารหลายประการ:

7.1 การควบคุมและกำหนดสถานะ (Control & Status Logic):

- **หน้าที่:** ควบคุมการไหลของข้อมูลและจัดการสถานะของอุปกรณ์ I/O ที่เชื่อมต่ออยู่.
- **รายละเอียด:** มีรีจิสเตอร์ (Registers) ภายใน I/O Module สำหรับ CPU เขียนคำสั่งควบคุม (Control Register) และอ่านสถานะของอุปกรณ์ (Status Register) เช่น "อุปกรณ์พร้อมใช้งานหรือไม่", "มีข้อมูลรออ่านหรือไม่", "เกิดข้อผิดพลาดหรือไม่". CPU จะใช้สัญญาณจาก Control Bus ในการอ่านและเขียนข้อมูลไปยังรีจิสเตอร์เหล่านี้.

7.2 บัฟเฟอร์ข้อมูล (Data Buffer Registers):

- **หน้าที่:** เป็นหน่วยความจำชั่วคราวขนาดเล็กภายใน I/O Module สำหรับเก็บข้อมูลที่กำลังจะรับเข้า (Input Buffer) หรือข้อมูลที่กำลังจะส่งออก (Output Buffer).
- **รายละเอียด:** เนื่องจากความเร็วในการทำงานของ CPU กับอุปกรณ์ I/O มักจะไม่เท่ากัน (CPU เร็วกว่ามาก) บัฟเฟอร์จึงช่วยรองรับความแตกต่างของความเร็วนี้ได้ ตัวอย่างเช่น เมื่อผู้ใช้พิมพ์ตัวอักษรบนคีย์บอร์ด ข้อมูลจะถูกเก็บไว้ใน Input Buffer ของ I/O Module ของคีย์บอร์ดก่อน จากนั้น CPU จึงจะอ่านข้อมูลจากบัฟเฟอร์นี้เมื่อพร้อม. ในทางกลับกัน เมื่อ CPU ต้องการส่งข้อมูลไปยังเครื่องพิมพ์ ก็จะเขียนข้อมูลลง Output Buffer ของ I/O Module ของเครื่องพิมพ์ แล้ว I/O Module จะค่อยๆ ส่งข้อมูลไปยังเครื่องพิมพ์ตามความเร็วที่เครื่องพิมพ์รองรับ.

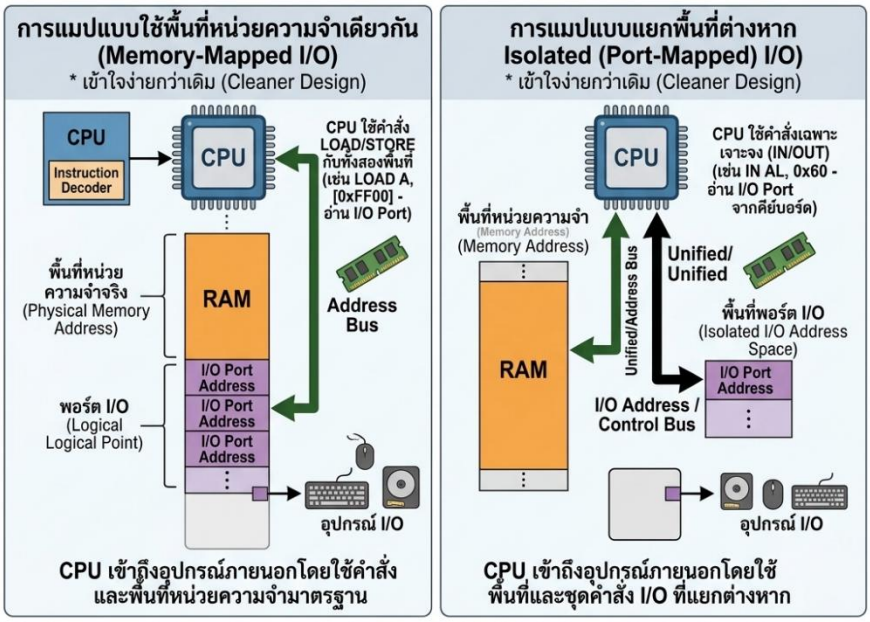
7.3 วงจรเข้ารหัสและถอดรหัสแอดเดรส (Address Decoding Logic):

- **หน้าที่:** ตรวจสอบสัญญาณแอดเดรสที่ส่งมาบน Address Bus เพื่อดูว่าแอดเดรสนั้นตรงกับแอดเดรสของ I/O Module ของตนเอง หรือพอร์ตใดพอร์ตหนึ่งภายใน I/O Module นั้นหรือไม่.
- **รายละเอียด:** I/O Module แต่ละตัว หรือแต่ละพอร์ตจะมีแอดเดรสเฉพาะตัว เมื่อ CPU ส่งแอดเดรสออกมา วงจรนี้จะทำการ "ถอดรหัส" และ "ตรวจสอบ" แอดเดรส หากตรงกัน I/O Module นั้นก็จะรับรู้ว่า CPU ต้องการสื่อสารกับตน.

7.4 วงจรจัดการการขัดจังหวะ (Interrupt Logic):

- **หน้าที่:** สร้างสัญญาณ Interrupt และส่งไปยัง CPU เมื่อมีเหตุการณ์สำคัญเกิดขึ้นกับอุปกรณ์ I/O ที่ต้องได้รับการตอบสนองจาก CPU (เช่น ข้อมูลพร้อม, เกิดข้อผิดพลาด).
- **รายละเอียด:** ช่วยลดภาระของ CPU ที่ไม่ต้องคอยตรวจสอบสถานะของอุปกรณ์ I/O ตลอดเวลา.

8.วิธีการโอนย้ายข้อมูลระบบบัส (System Bus) ผ่านพอร์ต (Port) และการทำงานของพอร์ต:



รูปที่ 5-13 สถาปัตยกรรมการจัดสรรเลขที่พอร์ต (I/O Port Addressing) และการแยกพื้นที่การติดต่อระหว่างหน่วยความจำกับอุปกรณ์ภายนอก

สรุป พอร์ต (Port) ในบริบทนี้คือ "จุดเชื่อมต่อเชิงตรรกะ" (Logical Connection Point) ภายใน I/O Module ที่ CPU ใช้อ้างอิงเพื่อสื่อสารกับอุปกรณ์ I/O ที่เฉพาะเจาะจง พอร์ตแต่ละพอร์ตจะมีแอดเดรสเฉพาะตัว (I/O Port Address) ซึ่ง CPU ใช้ในการระบุเป้าหมายของการสื่อสาร

วิธีการโอนย้ายข้อมูลผ่านพอร์ตหลักๆ:

8.1 การโอนย้ายข้อมูลโดยการแมปหน่วยความจำ (Memory-Mapped I/O):

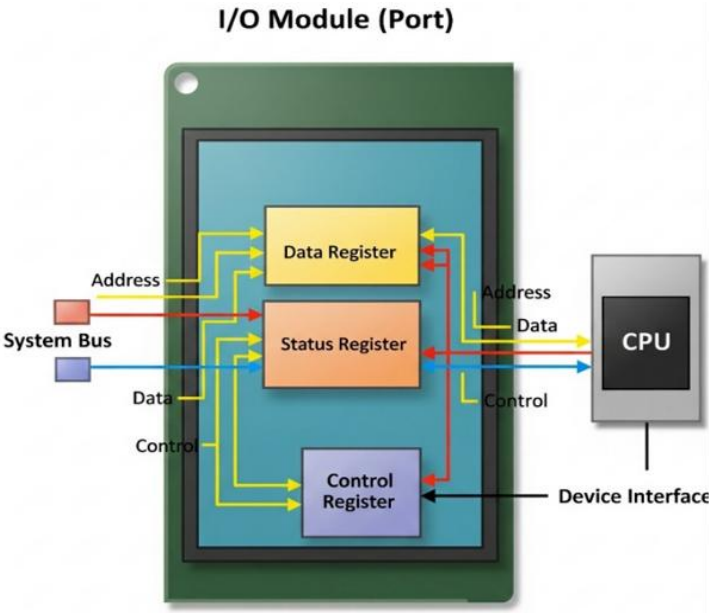
- **หลักการ:** พอร์ตของอุปกรณ์ I/O ถูกกำหนดให้มี "แอดเดรส" เหมือนตำแหน่งในหน่วยความจำหลัก.
- **การทำงาน:** CPU สามารถใช้คำสั่งที่ใช้ในการเข้าถึงหน่วยความจำ (เช่น LOAD, STORE) เพื่ออ่านหรือเขียนข้อมูลไปยังพอร์ต I/O ได้โดยตรง ระบบจะมองว่าพอร์ต I/O เป็นส่วนหนึ่งของพื้นที่แอดเดรสของหน่วยความจำ.
- **ข้อดี:** ไม่ต้องมีคำสั่ง I/O เฉพาะ ทำให้การเขียนโปรแกรมง่ายขึ้น.
- **ข้อเสีย:** ลดพื้นที่แอดเดรสที่ใช้สำหรับหน่วยความจำจริง และอุปกรณ์ I/O อาจจะถูกจำกัดความเร็วเท่ากับหน่วยความจำ.

8.2 การโอนย้ายข้อมูลโดยการแมปพอร์ตแยกต่างหาก (Isolated I/O / Port-Mapped I/O):

- **หลักการ:** พอร์ตของอุปกรณ์ I/O จะมีพื้นที่แอดเดรสของตัวเองที่ "แยกต่างหาก" จากพื้นที่แอดเดรสของหน่วยความจำ.

- **การทำงาน:** CPU จะต้องใช้คำสั่ง I/O เฉพาะ (เช่น IN สำหรับอ่าน, OUT สำหรับเขียน) ในการสื่อสารกับพอร์ต I/O สัญญาณควบคุมบน Control Bus จะแตกต่างกันระหว่างการเข้าถึงหน่วยความจำและการเข้าถึง I/O.
- **ข้อดี:** ไม่ลดพื้นที่แอดเดรสของหน่วยความจำ สามารถออกแบบ I/O Bus ให้มีความเร็วและคุณสมบัติแตกต่างจาก Memory Bus ได้.
- **ข้อเสีย:** ต้องมีคำสั่ง I/O เฉพาะ ทำให้การเขียนโปรแกรมซับซ้อนขึ้นเล็กน้อย.

9. บทบาทในการโอนย้ายข้อมูลของระบบบัส (System Bus) ไปยังพอร์ตต่างๆ (Port System): ไม่ว่าจะใช้การแมปแบบใด การทำงานของพอร์ตเป็นไปในลักษณะคล้ายกัน:



รูปที่ 5-14 สถาปัตยกรรมลำดับชั้นของระบบบัสประสิทธิภาพสูงและการเชื่อมต่ออุปกรณ์ผ่านโมดูลควบคุม (I/O Module) บนบัสส่วนขยาย

กรณีที่ 1 เมื่อ CPU ต้องการส่งข้อมูลไปยังอุปกรณ์ I/O (เช่น ส่งตัวอักษรไปจอภาพ):

1.CPU ส่งสัญญาณแอดเดรสของพอร์ตจอภาพ (ผ่าน Address Bus).

2.CPU ส่งข้อมูลตัวอักษร (ผ่าน Data Bus).

3.CPU ส่งสัญญาณควบคุม "Write to I/O" (ผ่าน Control Bus).

4.I/O Module ของจอภาพรับข้อมูลและคำสั่งไปเก็บใน Output Buffer และจัดการส่งไปยังจอภาพต่อไป.

กรณีที่ 2 เมื่อ CPU ต้องการรับข้อมูลจากอุปกรณ์ I/O (เช่น รับการกดแป้นจากคีย์บอร์ด):

1. ผู้ใช้กดแป้นบนคีย์บอร์ด, คีย์บอร์ดส่งสัญญาณข้อมูลไปยัง I/O Module ของคีย์บอร์ด.
2. I/O Module ของคีย์บอร์ดอาจเก็บข้อมูลไว้ใน Input Buffer และส่งสัญญาณ Interrupt ไปยัง CPU.
3. เมื่อ CPU รับ Interrupt และจัดการเรียบร้อย, CPU ส่งสัญญาณแอดเดรสของพอร์ตคีย์บอร์ด (ผ่าน Address Bus).
4. CPU ส่งสัญญาณควบคุม "Read from I/O" (ผ่าน Control Bus).
5. I/O Module ของคีย์บอร์ดนำข้อมูลจาก Input Buffer มาวางบน Data Bus เพื่อให้ CPU อ่านไปยังหน่วยความจำ

9.1 การทำงานของวงจรสื่อสารข้อมูลภายใน I/O Module: การทำงานไม่ใช่แค่ทางผ่านของข้อมูล แต่มีวงจรควบคุมและหน่วยความจำชั่วคราวเล็กๆ อยู่ภายใน เพื่อจัดการข้อมูลและสถานะต่างๆ โดยมีส่วนประกอบสำคัญดังนี้:

9.1.1 Bus Interface (ส่วนเชื่อมต่อบัสระบบ):

- เป็นส่วนที่เชื่อมต่อกับ **System Bus** ของคอมพิวเตอร์ ซึ่งประกอบด้วย **Address Bus, Data Bus และ Control Bus** ทำหน้าที่รับสัญญาณคำสั่ง, ที่อยู่, และข้อมูลจาก CPU และส่งข้อมูลกลับไปยัง CPU และ เป็นประตูทางเข้า-ออกหลักของ I/O Module ที่ติดต่อกับ CPU

9.1.2 Data Register (รีจิสเตอร์ข้อมูล): เป็นหน่วยความจำชั่วคราวขนาดเล็กภายใน I/O Module

- **หน้าที่:** ใช้พักข้อมูลที่กำลังจะถูกส่งไปยังอุปกรณ์ I/O (จาก CPU) หรือข้อมูลที่รับมาจากอุปกรณ์ I/O (เพื่อส่งต่อไปยัง CPU)
- **เปรียบเทียบ:** เหมือน "ห้องพักสินค้า" ชั่วคราว ก่อนที่สินค้า (ข้อมูล) จะถูกส่งออกไป หรือก่อนที่จะถูกนำไปเก็บที่คลังใหญ่ (CPU)

9.1.3 Status Register (รีจิสเตอร์สถานะ): เป็นหน่วยความจำชั่วคราวขนาดเล็กที่เก็บสถานะปัจจุบันของ I/O Module และอุปกรณ์ I/O ที่เชื่อมต่ออยู่

- **หน้าที่:** CPU สามารถอ่านค่าจาก Status Register เพื่อตรวจสอบว่าอุปกรณ์ I/O พร้อมใช้งานหรือไม่ (เช่น เครื่องพิมพ์พร้อมพิมพ์หรือกระดาษหมด, คีย์บอร์ดมีข้อมูลเข้ามาหรือไม่)
- **เปรียบเทียบ:** เหมือน "ป้ายบอกสถานะ" ว่าอุปกรณ์กำลังทำอะไรอยู่ หรือพร้อมที่จะรับ/ส่งข้อมูลแล้วหรือยัง

9.1.4 Control Register (รีจิสเตอร์ควบคุม): เป็นหน่วยความจำชั่วคราวขนาดเล็กที่เก็บ คำสั่งควบคุมการทำงานของ I/O Module และอุปกรณ์ I/O

- **หน้าที่:** CPU สามารถเขียนค่าลงใน Control Register เพื่อสั่งการให้อุปกรณ์ I/O ทำงานบางอย่าง (เช่น สั่งให้เครื่องพิมพ์เริ่มพิมพ์, สั่งให้ฮาร์ดดิสก์เริ่มอ่านข้อมูล)
- **เปรียบเทียบ:** เหมือน "แผงควบคุม" ที่ CPU ใช้กดปุ่มสั่งการให้อุปกรณ์ทำงาน

9.1.5 Device Interface (ส่วนเชื่อมต่ออุปกรณ์): เป็นส่วนที่เชื่อมต่อโดยตรงกับ **อุปกรณ์ I/O ภายนอก** (เช่น คีย์บอร์ด, จอภาพ)

- **หน้าที่** แปลงสัญญาณไฟฟ้าให้เหมาะสมกับอุปกรณ์ I/O นั้นๆ เนื่องจากอุปกรณ์แต่ละชนิดมีมาตรฐานการสื่อสารที่แตกต่างกัน

9.2 วิธีการโอนย้ายข้อมูลผ่านพอร์ต (Port) และการทำงานของพอร์ต

การโอนย้ายข้อมูลระหว่าง CPU กับอุปกรณ์ I/O ผ่าน I/O Module จะเกิดขึ้นเป็นขั้นตอน โดยใช้สัญญาณต่างๆ บน System Bus และการทำงานของรีจิสเตอร์ภายใน I/O Module:

ขั้นตอนที่ 1: CPU ตรวจสอบสถานะของอุปกรณ์ (Checking Status)

- CPU ส่ง **Address (ที่อยู่)** ของ **Status Register** ใน I/O Module นั้นๆ ไปบน **Address Bus**
- CPU ส่งสัญญาณ **Read** ไปบน **Control Bus**
- I/O Module จะส่งค่าสถานะจาก **Status Register** กลับไปให้ CPU ผ่าน **Data Bus**
- CPU จะอ่านค่าสถานะนี้เพื่อดูว่าอุปกรณ์ I/O พร้อมสำหรับการรับ/ส่งข้อมูลหรือไม่ (เช่น ถ้าเป็นคีย์บอร์ด จะดูว่ามีข้อมูลการกดปุ่มเข้ามาแล้วหรือยัง)

ขั้นตอนที่ 2: CPU สั่งการอุปกรณ์ (Sending Control Command)

- หาก CPU ต้องการสั่งให้อุปกรณ์ I/O ทำงาน (เช่น สั่งให้เครื่องพิมพ์เริ่มพิมพ์)
- CPU จะส่ง **Address (ที่อยู่)** ของ **Control Register** ใน I/O Module นั้นๆ ไปบน **Address Bus**
- CPU จะส่งสัญญาณ **Write** ไปบน **Control Bus**
- CPU จะส่ง **Data (ข้อมูลคำสั่ง)** ที่ต้องการสั่งไปบน **Data Bus**
- I/O Module จะรับข้อมูลคำสั่งนี้ไปเก็บไว้ใน **Control Register** และส่งต่อไปยัง **Device Interface** เพื่อสั่งให้อุปกรณ์ I/O ทำงานตามคำสั่งนั้น

ขั้นตอนที่ 3: การโอนย้ายข้อมูลจริง (Data Transfer)

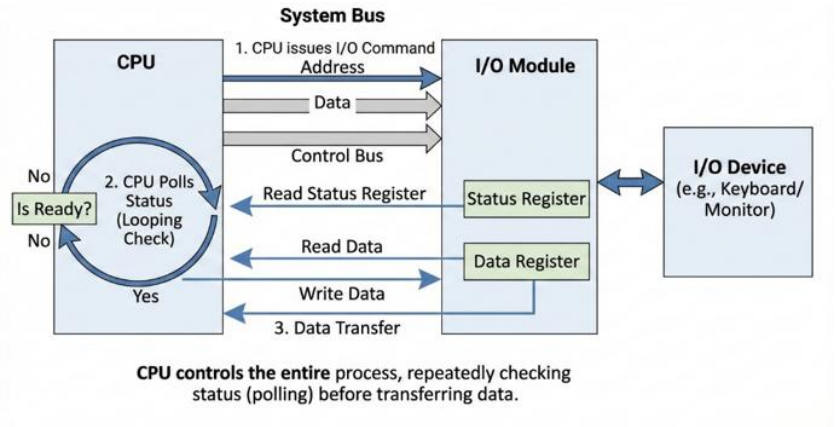
- **การเขียนข้อมูลจาก CPU ไปยังอุปกรณ์ I/O (Output):**
    1. CPU ตรวจสอบสถานะว่า I/O Module และอุปกรณ์พร้อมรับข้อมูลหรือไม่ (ผ่าน Status Register)
    2. CPU ส่ง **Address (ที่อยู่)** ของ **Data Register** ใน I/O Module นั้นๆ ไปบน **Address Bus**
    3. CPU ส่งสัญญาณ **Write** ไปบน **Control Bus**
    4. CPU ส่ง **Data (ข้อมูล)** ที่ต้องการเขียน (เช่น ข้อมูลภาพสำหรับจอภาพ) ไปบน **Data Bus**
    5. I/O Module จะรับข้อมูลนี้ไปเก็บไว้ใน **Data Register** ชั่วคราว จากนั้นจะส่งข้อมูลนี้ผ่าน **Device Interface** ไปยังอุปกรณ์ I/O ภายนอก
- **การอ่านข้อมูลจากอุปกรณ์ I/O ไปยัง CPU (Input):**

1. อุปกรณ์ I/O มีข้อมูลพร้อมส่ง (เช่น ข้อมูลจากการกดปุ่มคีย์บอร์ด) อาจส่ง **Interrupt Signal** ไปยัง CPU หรือ CPU อาจตรวจสอบสถานะเอง
2. CPU ตรวจสอบสถานะว่า I/O Module มีข้อมูลพร้อมให้อ่านหรือไม่ (ผ่าน Status Register)
3. CPU ส่ง **Address (ที่อยู่)** ของ **Data Register** ใน I/O Module นั้นๆ ไปบน **Address Bus**
4. CPU ส่งสัญญาณ **Read** ไปบน **Control Bus**
5. I/O Module จะส่งข้อมูลที่อยู่ใน **Data Register** (ซึ่งได้รับมาจากอุปกรณ์ I/O) กลับไปให้ CPU ผ่าน **Data Bus**

เพราะฉะนั้นสรุปได้ว่า I/O Module ทำหน้าที่เป็นเหมือน "ล่าม" และ "ผู้จัดการ" ที่คอยประสานงานการสื่อสารข้อมูลระหว่าง CPU กับอุปกรณ์ I/O ที่หลากหลาย ทำให้ระบบคอมพิวเตอร์ทำงานได้อย่างราบรื่นและมีประสิทธิภาพ

10. เทคนิคและวิธีการโอนย้ายข้อมูลระบบบัส (System Bus) ไปยังอุปกรณ์รับข้อมูลและแสดงผลข้อมูล (I/O Techniques)

การโอนย้ายข้อมูลระหว่างหน่วยประมวลผลกลาง (CPU) และอุปกรณ์ Input/Output (I/O Devices) มีหลายวิธีในการจัดการการโอนย้ายข้อมูลนี้ โดยแต่ละวิธีมีข้อดีข้อเสียและเหมาะกับสถานการณ์ที่แตกต่างกันไป วิธีการหลักๆ ที่ใช้กัน ได้แก่ Programmed I/O และ Interrupt Driven I/O



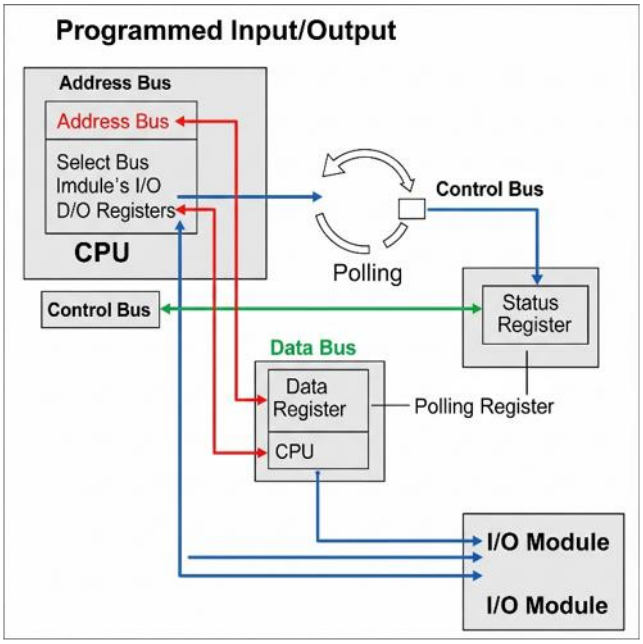
รูปที่ 5-15 ลำดับขั้นตอนการทำงานของเทคนิค Programmed I/O โดยใช้การตรวจสอบสถานะแบบ Polling

10.1 วิธีการโอนย้ายข้อมูลแบบโปรแกรม (Programmed I/O)

หลักการทำงานที่ CPU เป็นผู้โอนย้ายข้อมูลโดยตรง: วิธีการโอนย้ายข้อมูลแบบโปรแกรม (Programmed I/O) เป็นวิธีที่ง่ายที่สุดในการจัดการ I/O โดย **CPU เป็นผู้ควบคุมและจัดการการโอนย้ายข้อมูลทั้งหมดด้วยตัวเองอย่างสมบูรณ์** กล่าวคือ CPU จะต้องใช้คำสั่ง I/O ในโปรแกรม (เช่น IN, OUT หรือคำสั่ง LOAD/STORE ในกรณี Memory-Mapped I/O) เพื่ออ่านข้อมูลจากรีจิสเตอร์สถานะ (Status Register) ของ I/O Module เพื่อตรวจสอบว่าอุปกรณ์ I/O พร้อมที่จะรับหรือส่งข้อมูลหรือไม่ จากนั้นจึงจะอ่านหรือเขียนข้อมูลไปยังรีจิสเตอร์ข้อมูล (Data Register) ของ I/O Module นั้นๆ

กระบวนการทำงาน:

1. **CPU ส่งคำสั่ง I/O:** โปรแกรมที่กำลังทำงานบน CPU จะส่งคำสั่ง I/O ไปยัง I/O Module ของอุปกรณ์เป้าหมาย (เช่น "สั่งให้คีย์บอร์ดเตรียมส่งข้อมูล").
2. **CPU ตรวจสอบสถานะ (Polling):** CPU จะวนลูป (Loop) เพื่อ "ตรวจสอบ" (Poll) สถานะของ I/O Module อย่างต่อเนื่อง โดยการอ่านค่าจาก Status Register ซ้ำๆ เพื่อดูว่าอุปกรณ์พร้อมทำงานหรือมีข้อมูลพร้อมอ่านหรือไม่.
3. **การโอนย้ายข้อมูล:**
    - **สำหรับการอ่าน:** เมื่อ CPU ตรวจพบว่าข้อมูลพร้อม (เช่น มีการกดแป้นพิมพ์) CPU ก็จะส่งคำสั่งอ่าน (Read) และข้อมูลจาก I/O Module ก็จะถูกส่งผ่าน Data Bus เข้ามายัง CPU.
    - **สำหรับการเขียน:** เมื่อ CPU ต้องการเขียนข้อมูลออกไป (เช่น ส่งตัวอักษรไปจอภาพ) CPU จะตรวจสอบว่าอุปกรณ์พร้อมรับข้อมูลหรือไม่ ถ้าพร้อม CPU ก็จะส่งข้อมูลผ่าน Data Bus ออกไปยัง I/O Module.



รูปที่ 5-16 แผนผังแสดงความสัมพันธ์ระหว่าง CPU และ I/O Module ในกระบวนการ Programmed I/O ผ่านระบบบัส

จากรูปที่ 5-16 จะแสดง **"CPU"** และ **"I/O Module"** (ซึ่งเชื่อมต่อไปยังอุปกรณ์ I/O) รวมถึงมีเส้น **Address Bus, Data Bus, Control Bus** เชื่อมระหว่าง CPU และ I/O Module.

โดยจะมี "เส้นควบคุม" จาก CPU ไปยัง Status Register ภายใน I/O Module (แสดงการ Polling).

รวมถึงจะมี "เส้นข้อมูล" ระหว่าง CPU และ Data Register ภายใน I/O Module. และแสดงแผนผังการทำงานที่แสดง "Loop" ของ CPU ในการ Polling สถานะของอุปกรณ์อย่างต่อเนื่อง.

ปัญหาและข้อจำกัดของการโอนย้ายข้อมูลแบบ Programmed I/O:

1. **การสิ้นเปลืองเวลาของ CPU (CPU Wasting Time):** นี่คือปัญหาหลักที่สำคัญที่สุด CPU ต้องใช้เวลาส่วนใหญ่ในการ "รอ" และ "ตรวจสอบ" สถานะของอุปกรณ์ I/O อย่างต่อเนื่อง (Polling) ทำให้ CPU ไม่สามารถทำงานอื่นใดได้ในขณะที่รออุปกรณ์ I/O ซึ่งเป็นทรัพยากรที่มีค่าและประมวลผลได้เร็วมาก จึงเป็นการทำงานที่ไม่มีประสิทธิภาพอย่างยิ่ง.

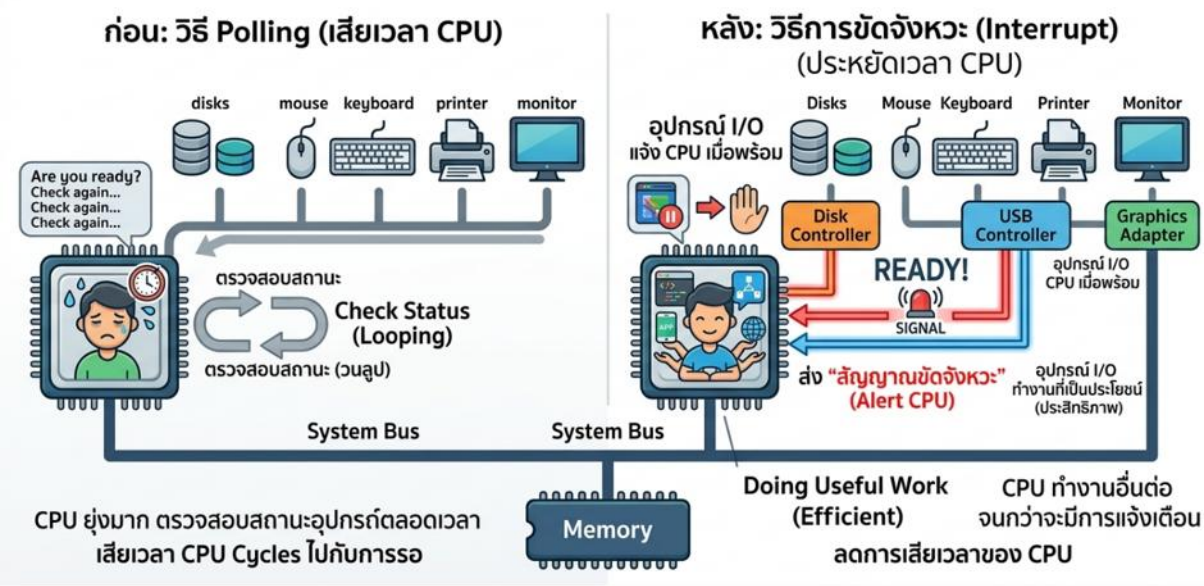
2. **ความเร็วในการทำงาน:** เหมาะสำหรับอุปกรณ์ I/O ที่มีความเร็วต่ำและมีการโอนย้ายข้อมูลน้อยๆ เท่านั้น เช่น คีย์บอร์ด แต่ไม่เหมาะกับอุปกรณ์ที่ต้องโอนย้ายข้อมูลจำนวนมากและรวดเร็ว เช่น ฮาร์ดดิสก์.
3. **การเขียนโปรแกรมซับซ้อนขึ้น:** นักพัฒนาโปรแกรมต้องเขียนโค้ดสำหรับ Polling และจัดการการตรวจสอบสถานะของ I/O เอง ซึ่งเพิ่มความซับซ้อนของโปรแกรม.

10.2 วิธีการโอนย้ายข้อมูลระบบบัส (System Bus) แบบอินเทอร์รัพท์ (Interrupt Driven I/O)

วิธีการโอนย้ายข้อมูลแบบอินเทอร์รัพท์ (Interrupt Driven I/O) ถูกพัฒนาขึ้นมาเพื่อ **แก้ไขปัญหาการสิ้นเปลืองเวลาของ CPU ที่เกิดจากการ Polling** ใน Programmed I/O โดยอาศัยกลไกการ "ขัดจังหวะ" การทำงานของ CPU

การแก้ปัญหาการเสียเวลาของ CPU โดยใช้วิธีการขัดจังหวะ:

แนวคิดหลักคือ CPU ไม่จำเป็นต้องคอยตรวจสอบสถานะของอุปกรณ์ I/O ตลอดเวลา แต่จะอนุญาตให้อุปกรณ์ I/O แจ้งให้ CPU ทราบเมื่อมีเหตุการณ์สำคัญเกิดขึ้นหรือพร้อมที่จะโอนย้ายข้อมูล โดยการส่ง **"สัญญาณขัดจังหวะ" (Interrupt Signal)** ไปยัง CPU แทน



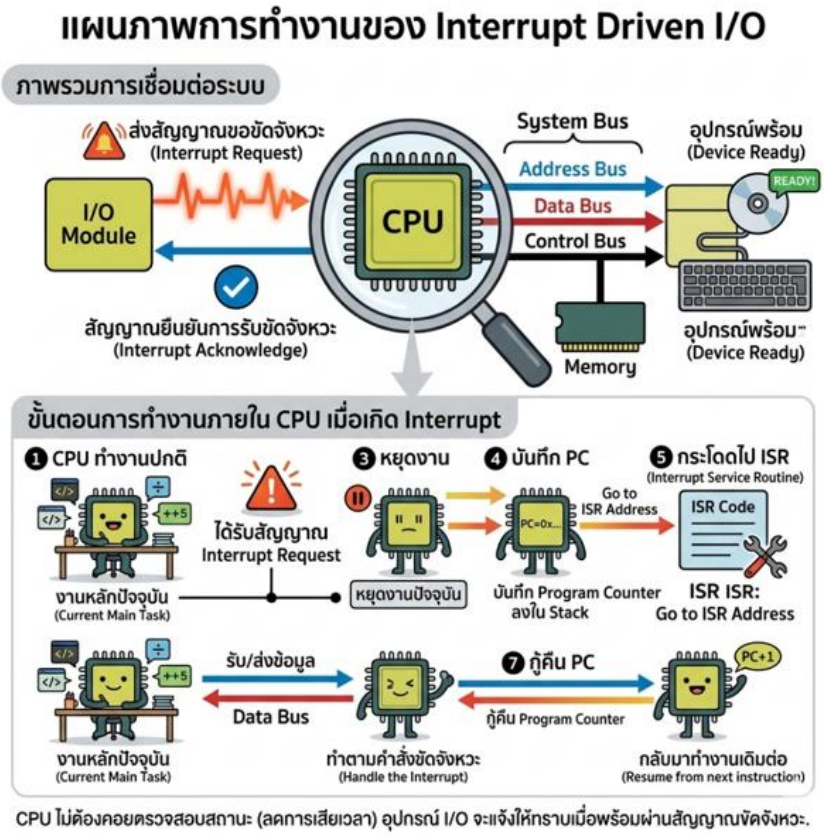
รูปที่ 5-17 การเปรียบเทียบกลไกและประสิทธิภาพระหว่างการติดต่อ I/O แบบ Polling และการใช้สัญญาณขัดจังหวะ (Interrupt-Driven I/O) เพื่อลดภาระการทำงานของ CPU

กระบวนการที่ CPU หยุดการทำงานปัจจุบันเพื่อไปบริการ Interrupt:

1. **อุปกรณ์ I/O ส่งสัญญาณ Interrupt:** เมื่ออุปกรณ์ I/O ทำงานเสร็จสิ้น (เช่น อ่านข้อมูลจากคีย์บอร์ดมาเก็บในบัฟเฟอร์แล้ว) หรือเกิดเหตุการณ์ที่ต้องได้รับการจัดการทันที I/O Module จะส่งสัญญาณ Interrupt ไปยัง CPU ผ่านสาย Interrupt บน Control Bus.

2. **CPU รับรู้ Interrupt:** CPU ที่กำลังประมวลผลงานอื่นอยู่จะรับรู้สัญญาณ Interrupt และเมื่อจบรอบคำสั่งปัจจุบัน (หรือจบรอบย่อยที่เหมาะสม) CPU จะหยุดการทำงานหลักชั่วคราว.
3. **บันทึกสถานะ CPU:** ก่อนที่จะไปบริการ Interrupt, CPU จะบันทึกสถานะปัจจุบันของตัวเองไว้ (เช่น ค่าใน Program Counter - PC เพื่อจำว่ากำลังทำคำสั่งไหนอยู่ และค่าใน Register ต่างๆ) ลงใน Stack ในหน่วยความจำ เพื่อให้สามารถกลับมาทำงานเดิมต่อได้อย่างถูกต้องหลังจากบริการ Interrupt เสร็จ.
4. **CPU กระโดดไปทำ Interrupt Service Routine (ISR):** CPU จะกระโดดไปทำงานในโปรแกรมย่อยพิเศษที่เรียกว่า **Interrupt Service Routine (ISR)** หรือ Interrupt Handler ซึ่งเป็นโปรแกรมที่เขียนขึ้นมาเพื่อจัดการกับ Interrupt ที่เกิดขึ้นจากอุปกรณ์นั้นๆ โดยเฉพาะ.
5. **บริการ Interrupt:** ISR จะดำเนินการที่จำเป็น เช่น อ่านข้อมูลจาก I/O Module หรือส่งข้อมูลออกไป, ตรวจสอบสถานะข้อผิดพลาด, หรือรีเซ็ตอุปกรณ์.
6. **CPU กลับมาทำงานเดิม:** เมื่อ ISR ทำงานเสร็จสิ้น CPU จะดึงสถานะที่บันทึกไว้ใน Stack กลับคืนมา และกลับไปทำงานในคำสั่งหลักที่ค้างไว้ก่อนหน้าการเกิด Interrupt.

รูปภาพที่ 5-18 จะแสดง **"CPU"** และ **"I/O Module"** (พร้อมอุปกรณ์ I/O). โดยมี **Address Bus, Data Bus, Control Bus** เชื่อมต่อ. **ในรูปสิ่งที่สำคัญที่สุดคือ:** เส้น **Interrupt Request** ที่พุ่งจาก I/O Module ไปยัง CPU และมีเส้น **Interrupt Acknowledge** ที่พุ่งกลับจาก CPU ไปยัง I/O Module. การทำงานของ CPU ที่ "หยุด" เมื่อเกิด Interrupt, "บันทึก PC", "กระโดดไป ISR", "ทำ ISR", "กู้คืน PC", "กลับมาทำงานเดิม".



รูปที่ 5-18 แผนภาพวงจรการทำงานของระบบ Interrupt Driven I/O และขั้นตอนการจัดการสัญญาณขัดจังหวะภายใน CPU

11.วงจรการควบคุมระบบบัส (System Bus) ผ่านการอินเทอร์รัพท์คอนโทลเลอร์ (Interrupt Controller):

ในระบบที่มีอุปกรณ์ I/O จำนวนมาก แต่ละอุปกรณ์อาจส่งสัญญาณ Interrupt ได้ ซึ่งอาจทำให้เกิด Interrupt พร้อมกันหลายตัวได้ เพื่อจัดการสถานการณ์นี้และลดภาระของ CPU จึงมีฮาร์ดแวร์พิเศษที่เรียกว่า **Interrupt Controller** (เช่น Programmable Interrupt Controller - PIC)

IRQ	Usage
0	system timer (cannot be changed)
1	keyboard controller (cannot be changed)
2	cascaded signals from IRQs 8–15
3	second RS-232 serial port (COM2: in Windows)
4	first RS-232 serial port (COM1: in Windows)
5	parallel port 2 and 3 or sound card
6	floppy disk controller
7	first parallel port
8	real-time clock
9	open interrupt
10	open interrupt
11	open interrupt
12	PS/2 mouse
13	math coprocessor
14	primary ATA channel
15	secondary ATA channel

รูปที่ 5-19 ตารางมาตรฐานหมายเลขขัดจังหวะ (IRQ Mapping) ระหว่างสถาปัตยกรรม Intel 8088

- **หน้าที่ในการจัดการสัญญาณ:** Interrupt Controller จะทำหน้าที่รวบรวมและรับสัญญาณ Interrupt จากอุปกรณ์ I/O หลายๆ ตัว.
- **การส่งสัญญาณไปยัง CPU:** เมื่อ Interrupt Controller ได้รับสัญญาณ Interrupt จากอุปกรณ์อย่างน้อยหนึ่งตัว มันจะส่งสัญญาณ Interrupt เพียงตัวเดียวไปยัง CPU.
- **การจัดลำดับความสำคัญของ Interrupt (Priority Handling):** หากมี Interrupt หลายตัวเกิดขึ้นพร้อมกัน Interrupt Controller จะมีกลไกในการ "จัดลำดับความสำคัญ" (Priority) และส่ง Interrupt ที่มีความสำคัญสูงที่สุดไปยัง CPU ก่อน. จากนั้นเมื่อ CPU จัดการ Interrupt นั้นเสร็จ Interrupt Controller จะส่ง Interrupt ตัวถัดไปที่มีความสำคัญรองลงมา.

จากรูปที่ 5-20 ตารางนี้แสดงให้เห็นถึงการจัดการ Interrupt ที่มีความซับซ้อนและประสิทธิภาพสูงขึ้น

1.APIC Vector Mapping: การเปลี่ยนจากระบบ IRQ แบบเก่า (Legacy) มาเป็นระบบเวกเตอร์ที่ระบุตำแหน่งได้แม่นยำขึ้น

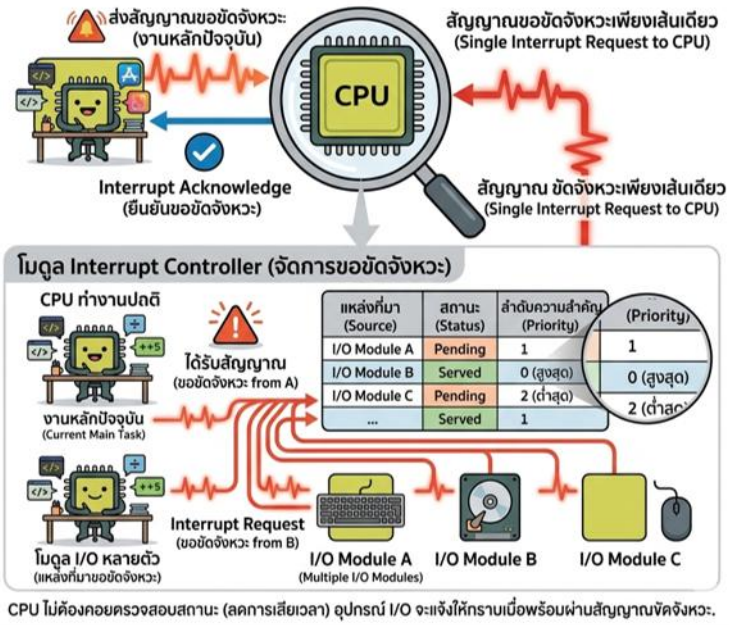
2.Modern Device Support: มีการจัดสรรช่วง IRQ เฉพาะสำหรับเทคโนโลยีใหม่ เช่น PCIe Port Controllers, NVMe Drive, USB 3.x Host, และ Thunderbolt 4

3.Networking & Connectivity: แสดงการแยกแยะสัญญาณขัดจังหวะสำหรับ Wi-Fi 6E/7 และ Bluetooth เพื่อลดการรบกวนกันของสัญญาณ

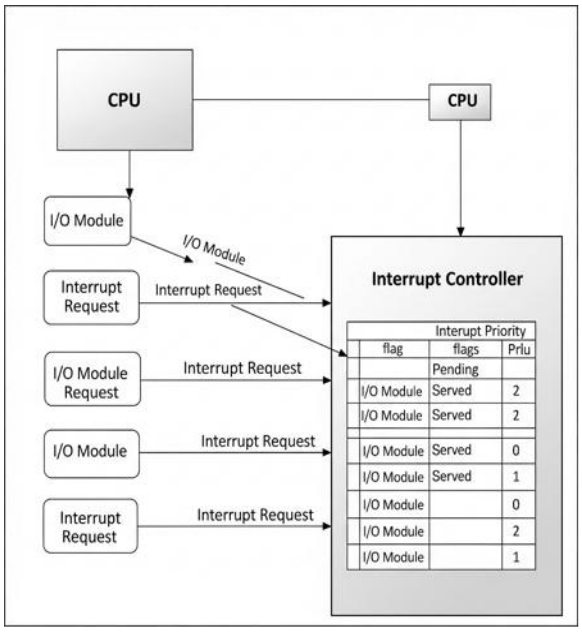
Contemporary Intel CPU APIC IRQ Mapping (Example for Modern Desktop/Server)	
IRQ	**Modern Usage (APIC Vector Mapping)**
0 - 1	**Reserved** (Internal CPU Timers, Internal Keyboard Logic)
2 - 15	**Reserved Range** (Internal CPU Handlers, Platform Control)
16 - 23	**PCIe Port 1 Controller** (Main NVMe Drive, Graphics Controller Interrupt)
24 - 31	**PCIe Port 2 Controller** (Secondary M.2, Discrete Audio)
32 - 39	**USB 3.x Host Controller** (Type-C Docking, Modern Keyboard/Mouse via XHCI)
44 - 55	**SATA Host Controller** (High-Performance Optical Drive, SATA SSD)
56 - 63	**Integrated Graphics** (Intel UHD/Xe Graphics Unit)
64 - 71	**Thunderbolt 4 / USB4 Controllers** (High-Bandwidth Docks/Devices)
72 - 79	**Wi-Fi 6E/7 Controller** (Multi-Stream Wireless Network)
80 - 87	**Bluetooth Controller** (Advanced BLE/Classic Host)
88 - 95	**Modern Embedded Controller (EC)** (Platform Power Management)
96 - 103	**PCI Express Root Complex** (Internal Bus Arbitration)
104 - 111	**Legacy Compatible Device 1** (Logical Mapping)
112 - 119	Reserved Range
120+	**MSI-X and IPI Vector Mapping Range** (Multicore CPU-to-CPU Signaling and

รูปที่ 5-20 ตารางหมายเลขเวกเตอร์ขัดจังหวะ (IRQ Mapping) สำหรับ CPU Intel ระบบคอมพิวเตอร์ยุคปัจจุบัน

4.Multicore Optimization: ช่วง IRQ ตั้งแต่ 120+ ถูกกำหนดไว้สำหรับ MSI-X และ IPI (Inter-Processor Interrupt) ซึ่งจำเป็นมากสำหรับการสื่อสาร**ระหว่างคอร์ใน CPU รุ่นปัจจุบัน**



รูปที่ 5-21 สถาปัตยกรรมการจัดการสัญญาณขัดจังหวะผ่านโมดูลควบคุม (Interrupt Controller) และการจัดลำดับความสำคัญสำหรับอุปกรณ์ I/O หลายตัว



รูปที่ 5-22 โครงสร้างการเชื่อมต่อสัญญาณขัดจังหวะจากหลายโมดูล I/O เข้าสู่ Interrupt Controller และการจัดลำดับความสำคัญ

จากรูปที่ 5-21 และ 5-22 **"CPU"**.จะมีโมดูล **"Interrupt Controller"** อยู่ระหว่าง CPU กับ **"I/O Modules หลายๆ ตัว"**.และมีเส้น Interrupt Request จากแต่ละ I/O Module จะพุ่งเข้าหา Interrupt Controller.จากนั้นมีเพียงเส้น Interrupt Request เพียงเส้นเดียวที่ออกจาก Interrupt Controller ไปยัง CPU.และตัวโมดูลจะมี ตารางภายใน Interrupt Controller ที่เก็บสถานะและลำดับความสำคัญของ Interrupt ต่างๆ.

11.1ชนิดของอินเทอร์รัพท์: Interrupt สามารถแบ่งออกได้หลายประเภทตามลักษณะและที่มา:

11.1.1 Nonmaskable Interrupt (NMI):

- **รายละเอียด:** เป็น Interrupt ที่ไม่สามารถถูก "ละเว้น" หรือ "ปิดกั้น" ได้โดย CPU เป็น Interrupt ที่มีความสำคัญสูงสุดและ CPU ต้องตอบสนองทันที ไม่ว่าจะเกิดอะไรขึ้นก็ตาม.
- **ตัวอย่าง:** มักใช้สำหรับเหตุการณ์วิกฤต เช่น ความผิดพลาดของหน่วยความจำ (Memory Parity Error) หรือปัญหาฮาร์ดแวร์ร้ายแรง.

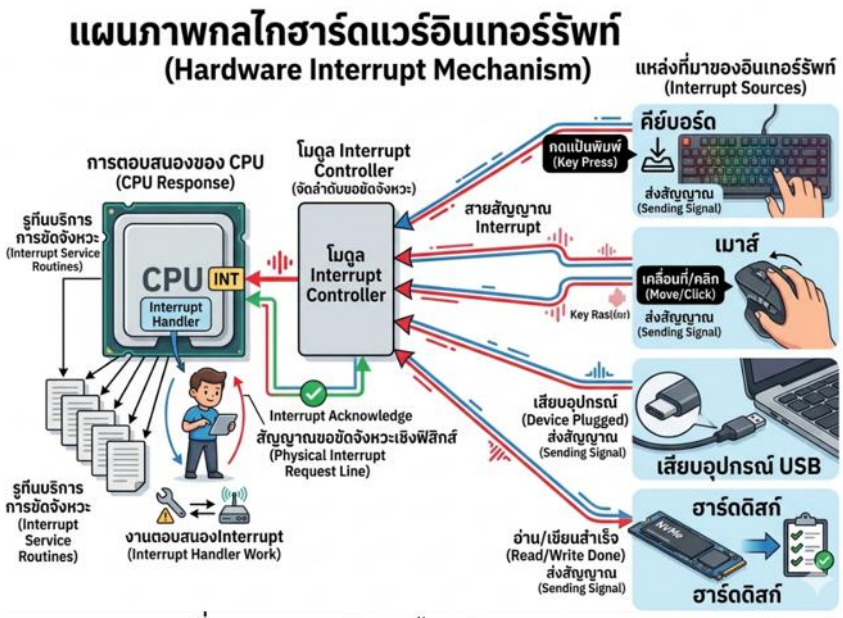
11.1.2 Maskable Interrupt (MI):

- **รายละเอียด:** เป็น Interrupt ที่ CPU สามารถ "ปิดกั้น" หรือ "ละเว้น" ได้ชั่วคราว โดยการตั้งค่าใน Status Register ของ CPU เมื่อ CPU กำลังทำงานที่สำคัญและไม่ต้องการให้ถูก

ขัดจังหวะ CPU สามารถ Mask (ปิดกั้น) Interrupt ชนิดนี้ได้ และจะอนุญาตให้ทำงานอีกครั้งเมื่อพร้อม.

- **ตัวอย่าง:** Interrupt จากอุปกรณ์ I/O ส่วนใหญ่ เช่น คีย์บอร์ด, เมาส์, ดิสก์.

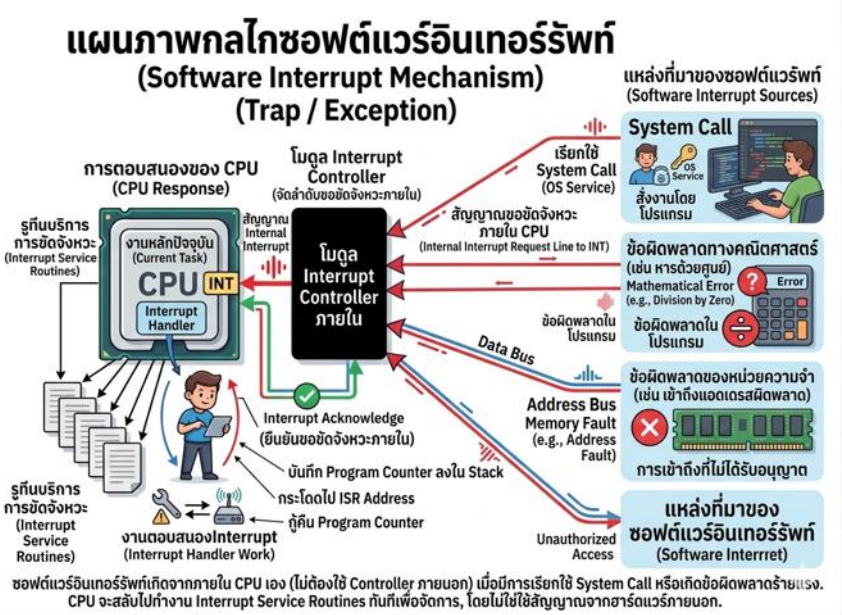
11.2 Hardware Interrupt:



รูปที่ 5-23 แผนภาพกลไก Hardware Interrupt

- **รายละเอียด:** Interrupt ที่เกิดจากอุปกรณ์ฮาร์ดแวร์ภายนอก CPU (เช่น I/O Devices) ส่งสัญญาณทางกายภาพผ่านสายสัญญาณ Interrupt ไปยัง CPU หรือ Interrupt Controller.
- **ตัวอย่าง:** การกดแป้นพิมพ์, การเคลื่อนเมาส์, การเสียบ USB, ฮาร์ดดิสก์อ่าน/เขียนข้อมูลเสร็จ.

11.2.1 Software Interrupt (หรือ Trap / Exception):



รูปที่ 5-24 แผนภาพกลไก Software Interrupt

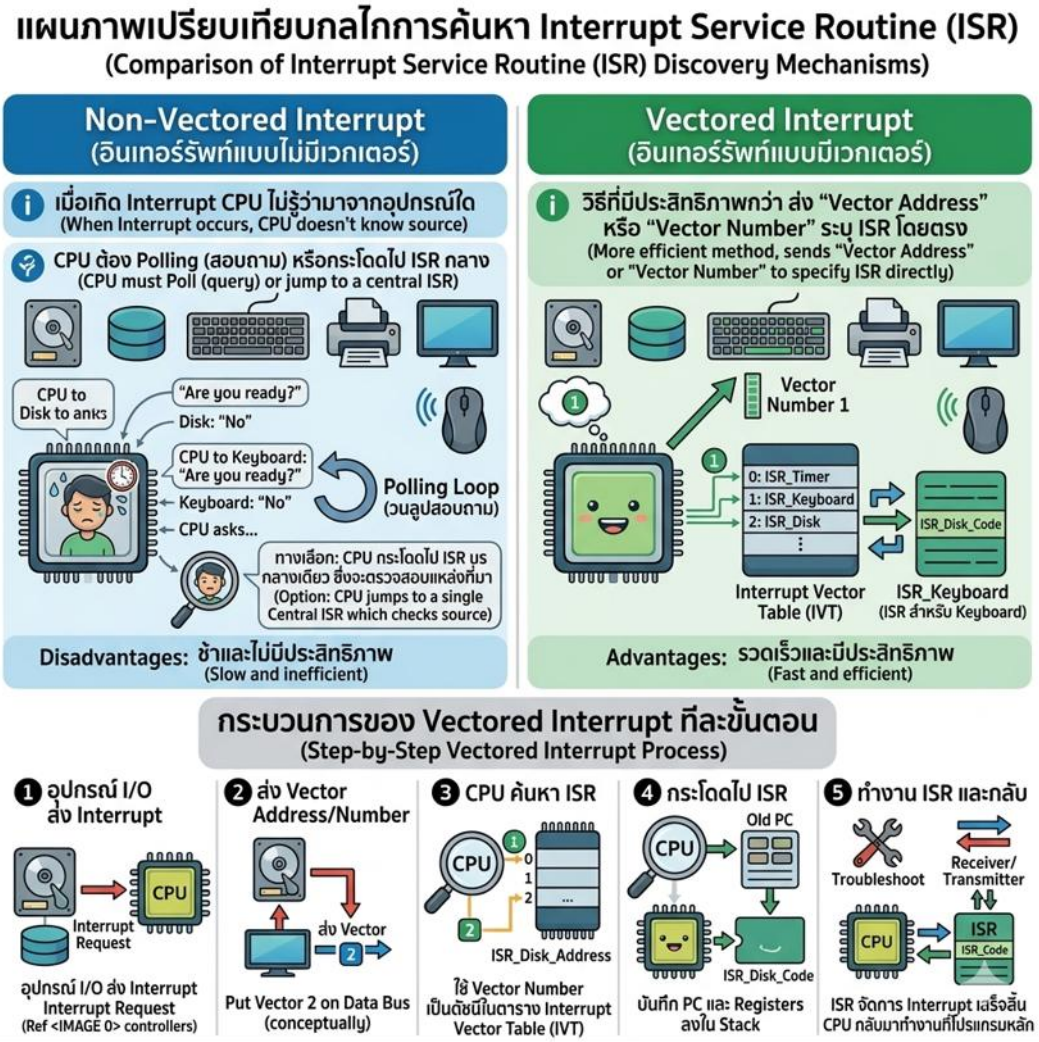
- **รายละเอียด:** Interrupt ที่เกิดจากการทำงานของซอฟต์แวร์เองโดยไม่ใช่สัญญาณจากฮาร์ดแวร์ภายนอก อาจเกิดจากการสั่งงานโดยโปรแกรม (เช่น การเรียกใช้ System Call) หรือเกิดจากข้อผิดพลาดในโปรแกรม (เช่น การหารด้วยศูนย์, การเข้าถึงหน่วยความจำที่ไม่ได้รับอนุญาต).
- **ตัวอย่าง:** การเรียกใช้บริการจากระบบปฏิบัติการ (OS), การเกิดข้อผิดพลาดทางคณิตศาสตร์, การเข้าถึงแอดเดรสผิดพลาด.

11.3 Non-Vectored Interrupt และ Vector Interrupt รวมถึงกระบวนการของ Vectored Interrupt เทียบกับการทำงานจริง

เมื่อเกิด Interrupt ขึ้น CPU จำเป็นต้องทราบว่าจะต้องกระโดดไปทำ Interrupt Service Routine (ISR) ตัวไหน เพื่อจัดการกับ Interrupt นั้นๆ วิธีการที่ CPU ค้นหา ISR มีสองแบบ:

11.3.1 Non-Vectored Interrupt:

- **หลักการ:** เมื่อเกิด Interrupt ขึ้น CPU จะทราบเพียงว่ามี Interrupt เกิดขึ้น แต่ไม่รู้ว่ามาจากอุปกรณ์ใด.
- **กระบวนการ:** CPU จะต้อง "Polling" (สอบถาม) I/O Module ทีละตัวว่าใครเป็นผู้ส่ง Interrupt มา ซึ่งอาจทำให้เสียเวลา. หรือ CPU อาจจะกระโดดไปยัง ISR กลางเพียงตัวเดียว จากนั้น ISR กลางนี้จะทำหน้าที่ตรวจสอบว่า Interrupt มาจากไหนแล้วค่อยเรียก ISR ที่ถูกต้องอีกที.
- **ข้อเสีย:** ช้าและไม่มีประสิทธิภาพ โดยเฉพาะเมื่อมีอุปกรณ์ I/O จำนวนมาก.



รูปที่ 5-25 แผนภาพเปรียบเทียบกลไกการค้นหา Interrupt service Routine (ISR)

11.3.2 Vectored Interrupt:

- **หลักการ:** เป็นวิธีที่มีประสิทธิภาพกว่า เมื่อเกิด Interrupt ขึ้น I/O Module (หรือ Interrupt Controller) จะส่ง "เวกเตอร์แอดเดรส" (Vector Address) หรือ "เวกเตอร์หมายเลข" (Vector Number) ซึ่งเป็นค่าที่ใช้ระบุตำแหน่งเริ่มต้นของ ISR ที่เกี่ยวข้องโดยตรง.
- **กระบวนการของ Vectored Interrupt:**
  1. **อุปกรณ์ I/O ส่ง Interrupt:** อุปกรณ์ I/O หรือ I/O Module ที่เกิดเหตุการณ์จะส่งสัญญาณ Interrupt ไปยัง CPU (อาจผ่าน Interrupt Controller).
  2. **ส่ง Vector Address:** เมื่อ CPU ตอบรับ (Interrupt Acknowledge) I/O Module นั้นจะวาง "Vector Number" หรือ "Vector Address" ลงบน Data Bus (หรือ Bus อื่นๆ ที่ออกแบบไว้).
  3. **CPU ค้นหา ISR:** CPU จะรับ Vector Number นั้นไป และใช้มันเป็นดัชนี (Index) ในตารางที่เรียกว่า "Interrupt Vector Table" (IVT) ซึ่งตารางนี้จะเก็บตำแหน่งเริ่มต้นของ ISR ของอุปกรณ์แต่ละตัว.
  4. **กระโดดไป ISR:** เมื่อได้แอดเดรสของ ISR ที่ถูกต้องจาก IVT แล้ว CPU จะบันทึกสถานะของตัวเอง (PC, Registers) ลง Stack แล้วกระโดดไปเริ่มทำงานที่ ISR นั้นทันที.
  5. **ทำงาน ISR และกลับ:** ISR จัดการ Interrupt เสร็จสิ้น และ CPU กลับมาทำงานที่โปรแกรมหลัก
- **ข้อดี:** รวดเร็วและมีประสิทธิภาพ เพราะ CPU ทราบทันทีว่าต้องไปบริการ Interrupt ใดโดยไม่ต้องเสียเวลา Polling.

11.4 ขั้นตอนการทำงานของ Vectored Interrupt เปรียบเทียบกับการทำงานจริง:

สมมติว่าคุณกำลังพิมพ์งานอยู่ และต้องการบันทึกไฟล์ลงฮาร์ดดิสก์:

11.4.1 I/O Device ส่ง Interrupt Request:

- เมื่อคุณกดปุ่มบนคีย์บอร์ด (I/O Device) ตัวคีย์บอร์ดจะสร้างสัญญาณ **Interrupt Request** ส่งไปยัง **Interrupt Controller**
- หรือเมื่อฮาร์ดดิสก์อ่านข้อมูลเสร็จแล้ว ฮาร์ดดิสก์ก็จะส่งสัญญาณ **Interrupt Request** ไปยัง Interrupt Controller เช่นกัน
- **เปรียบเทียบ:** เหมือนอุปกรณ์ I/O "กดกริ่ง" เพื่อเรียกความสนใจจากผู้จัดการ (Interrupt Controller)

11.4.2 Interrupt Controller จัดการและส่ง Interrupt Request ไปยัง CPU:

- Interrupt Controller จะรับสัญญาณ Interrupt Request จากอุปกรณ์ I/O หลายๆ ตัว
- หากมี Interrupt หลายตัวพร้อมกัน Interrupt Controller จะใช้ตารางลำดับความสำคัญ (Priority) ภายในตัวมันเอง เพื่อตัดสินใจว่าจะส่ง Interrupt ของอุปกรณ์ใดไปให้ CPU ก่อน
- จากนั้น Interrupt Controller จะส่งสัญญาณ **Interrupt Request** เพียงเส้นเดียวไปยัง CPU
- **เปรียบเทียบ:** ผู้จัดการ (Interrupt Controller) จะรับกริ่งจากหลายๆ ห้องพร้อมกัน แล้วจัดคิวว่าห้องไหนสำคัญกว่า แล้วค่อยไปเคาะประตูเรียก CPU

11.4.3 CPU รับ Interrupt Request และส่ง Interrupt Acknowledge:

- เมื่อ CPU ได้รับสัญญาณ Interrupt Request จาก Interrupt Controller
- CPU จะหยุดการทำงานปัจจุบันชั่วคราว (เช่น หยุดประมวลผลโปรแกรมที่คุณกำลังพิมพ์อยู่)
- CPU จะส่งสัญญาณ **Interrupt Acknowledge** กลับไปยัง Interrupt Controller เพื่อบอกว่า "ฉันได้รับสัญญาณขัดจังหวะแล้วนะ"
- **เปรียบเทียบ:** CPU หยุดงานที่ทำอยู่ แล้วตอบกลับผู้จัดการว่า "โอเค ได้ยินแล้ว มีอะไร?"

11.4.4 I/O Module/Controller ส่ง Vector Number ผ่าน Data Bus:

- เมื่อ Interrupt Controller ได้รับสัญญาณ Interrupt Acknowledge จาก CPU
- Interrupt Controller จะส่ง "รหัสเฉพาะ" ที่เรียกว่า **Vector Number** ของอุปกรณ์ I/O ที่เป็นต้นเหตุของ Interrupt นั้นๆ ไปยัง CPU
- **สำคัญคือ:** Vector Number นี้จะถูกส่งผ่าน **Data Bus** ไม่ใช่ Address Bus หรือ Control Bus
- **เปรียบเทียบ:** ผู้จัดการ (Interrupt Controller) จะส่ง "นามบัตร" หรือ "รหัสประจำตัว" ของห้องที่กดกริ่งมาให้ CPU โดยส่งผ่าน "ช่องรับส่งเอกสาร" (Data Bus)

11.4.5 CPU ใช้ Vector Number เพื่อหา ISR ใน Interrupt Vector Table (IVT):

- เมื่อ CPU ได้รับ Vector Number
- CPU จะใช้ Vector Number นี้เป็น "ดัชนี" (Index) เพื่อไปค้นหา "ที่อยู่เริ่มต้น" ของโปรแกรมบริการ Interrupt (ISR) ที่เหมาะสมใน **Interrupt Vector Table (IVT)** ซึ่งอยู่ในหน่วยความจำ (RAM)
- **เปรียบเทียบ:** CPU รับนามบัตร (Vector Number) แล้วเปิด "สมุดโทรศัพท์ฉุกเฉิน" (IVT) เพื่อหาเบอร์โทรศัพท์ (ที่อยู่ของ ISR) ของผู้เชี่ยวชาญที่จะมาจัดการกับปัญหาจากนามบัตรนั้น

11.4.6 CPU กระโดดไปรัน ISR:

- เมื่อ CPU ได้ที่อยู่ของ ISR ที่ถูกต้องจาก IVT แล้ว
- CPU จะบันทึกสถานะการทำงานปัจจุบัน (เช่น ตำแหน่งคำสั่งที่กำลังทำอยู่) ไว้ใน Stack (หน่วยความจำชั่วคราว) เพื่อให้กลับมาทำงานเดิมได้หลังจากจัดการ Interrupt เสร็จ
- จากนั้น CPU จะ "กระโดด" ไปยังที่อยู่เริ่มต้นของ ISR นั้น และเริ่มประมวลผลคำสั่งใน ISR
- **เปรียบเทียบ:** CPU พักงานที่ทำอยู่ จดไว้ว่าทำอะไรค้างไว้ แล้วรีบวิ่งไปหาผู้เชี่ยวชาญ (ISR) เพื่อให้มาจัดการปัญหาเฉพาะหน้า (เช่น รับข้อมูลจากการกดปุ่มคีย์บอร์ด, บันทึกข้อมูลลงฮาร์ดดิสก์)

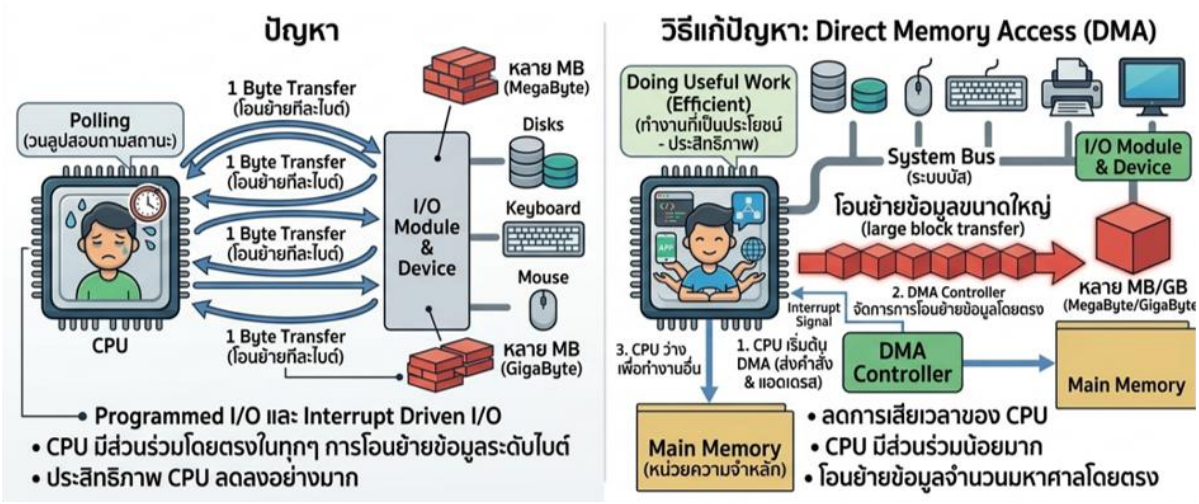
11.4.8 ISR ทำงานเสร็จสิ้นและ CPU กลับมาทำงานเดิม:

- เมื่อ ISR ทำงานเสร็จสิ้น (เช่น จัดการข้อมูลจากคีย์บอร์ดเรียบร้อย)
- ISR จะส่งสัญญาณบอก CPU ว่าเสร็จแล้ว
- CPU จะกู้คืนสถานะการทำงานที่บันทึกไว้ และกลับมาประมวลผลคำสั่งเดิมที่ทำค้างไว้ก่อนที่จะถูกขัดจังหวะ
- **เปรียบเทียบ:** ผู้เชี่ยวชาญ (ISR) จัดการปัญหาเสร็จแล้ว CPU ก็กลับมาทำงานหลักที่ทำค้างไว้

11.5 ประโยชน์ของ Vectored Interrupt:

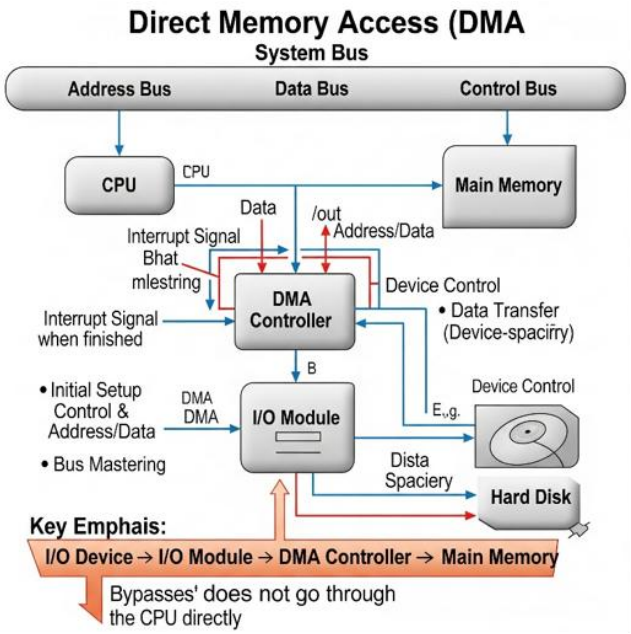
- **ประสิทธิภาพสูง:** CPU ไม่ต้องเสียเวลา Polling ทำให้มีเวลาไปประมวลผลงานหลักมากขึ้น
- **ตอบสนองรวดเร็ว:** เมื่อเกิด Interrupt CPU สามารถระบุ ISR ที่ถูกต้องได้ทันที
- **จัดการง่าย:** Interrupt Controller ช่วยจัดการ Interrupt จากอุปกรณ์หลายตัว ทำให้ CPU ไม่ต้องรับภาระมากเกินไป

12. การควบคุมระบบบัส (System Bus) เพื่อโอนย้ายข้อมูลกับหน่วยความจำโดยตรง (Direct Memory Access - DMA)



รูปที่ 5-26 การเปรียบเทียบข้อจำกัดของการโอนย้ายข้อมูลผ่าน CPU และการเพิ่มประสิทธิภาพระบบด้วยเทคนิคการเข้าถึงหน่วยความจำโดยตรง DMA

การโอนย้ายข้อมูลแบบโปรแกรม (Programmed I/O) และแบบอินเทอร์รัปต์ (Interrupt Driven I/O) ที่ได้อธิบายมีข้อจำกัดที่สำคัญคือ **CPU ยังคงต้องมีส่วนร่วมโดยตรง** ในทุกๆ การโอนย้ายข้อมูลระดับไบต์ (byte-by-byte transfer) ซึ่งหากเป็นการโอนย้ายข้อมูลจำนวนมหาศาล (เช่น หลายเมกะไบต์หรือกิกะไบต์) ก็จะทำให้ CPU ต้องใช้เวลาส่วนใหญ่ไปกับการจัดการการโอนย้ายข้อมูล ทำให้ประสิทธิภาพในการประมวลผลงานอื่นๆ ของ CPU ลดลงอย่างมากเพื่อแก้ไขปัญหานี้ จึงได้มีการพัฒนาเทคนิคที่เรียกว่า **Direct Memory Access (DMA)** ขึ้นมา



รูปที่ 5-27 การเชื่อมต่อระหว่าง CPU, DMA Controller และหน่วยความจำหลัก เพื่อการโอนย้ายข้อมูลโดยไม่ผ่านตัวประมวลผล

กระบวนการ DMA เป็นวิธีการโอนย้ายข้อมูลที่อนุญาตให้ I/O Module **(หรืออุปกรณ์ I/O ที่มี DMA Controller ในตัว) สามารถโอนถ่ายข้อมูลไปยังหน่วยความจำหลัก (Main Memory) ได้โดยตรง หรือจากหน่วยความจำหลักมายัง I/O Module ได้โดยตรง โดยไม่จำเป็นต้องผ่าน CPU ในทุกๆ ไบต์ของการโอนย้าย** กล่าวคือ I/O Module จะเข้าถึง Main Memory ได้ด้วยตัวเอง โดยใช้บัสระบบ (System Bus) ร่วมกับ CPU แต่ไม่ต้องให้ CPU เป็นผู้สั่งการและควบคุมการโอนย้ายข้อมูลทีละไบต์

12.1 เหตุผลที่ใช้ DMA สำหรับการโอนย้ายข้อมูลจำนวนมาก (เช่น Hard disk):

12.1.1 เพื่อลดภาระ CPU (CPU Offloading): CPU ไม่ต้องเสียเวลาในการควบคุมการโอนย้ายข้อมูลทีละไบต์ ทำให้สามารถนำเวลาไปประมวลผลงานอื่นๆ ได้อย่างเต็มที่ เพิ่มประสิทธิภาพโดยรวมของระบบ.

12.1.2 เพื่อเพิ่มความเร็วในการโอนย้ายข้อมูล: การโอนย้ายข้อมูลโดยตรงระหว่าง I/O Module กับ Memory จะเร็วกว่าการที่ต้องผ่าน CPU ในทุกๆ ขั้นตอน ทำให้เหมาะสำหรับอุปกรณ์ที่มีอัตราการโอนย้ายข้อมูลสูง (High Data Transfer Rate).

12.1.3 เพื่อความเหมาะกับอุปกรณ์บล็อก (Block Devices): อุปกรณ์อย่างฮาร์ดดิสก์ (Hard Disk Drive - HDD) หรือ Solid State Drive (SSD) ทำงานกับการโอนย้ายข้อมูลเป็น "บล็อก" (Block) หรือ "กลุ่ม" (Chunk) ที่มีขนาดใหญ่ (เช่น เป็นหลายกิโลไบต์หรือเมกะไบต์ในแต่ละครั้ง) DMA จึงเหมาะสมอย่างยิ่งในการโอนย้ายข้อมูลบล็อกเหล่านี้.

12.2 บทบาทของ CPU ในกระบวนการ DMA: การเริ่มต้นและรับทราบเมื่อการโอนย้ายเสร็จสิ้น:
กระบวนการ DMA จะช่วยให้ CPU ไม่ต้องควบคุมการโอนย้ายข้อมูลโดยตรง แต่ CPU ก็ยังคงมีบทบาทสำคัญ

12.2.1 การเริ่มต้น (Initialization):

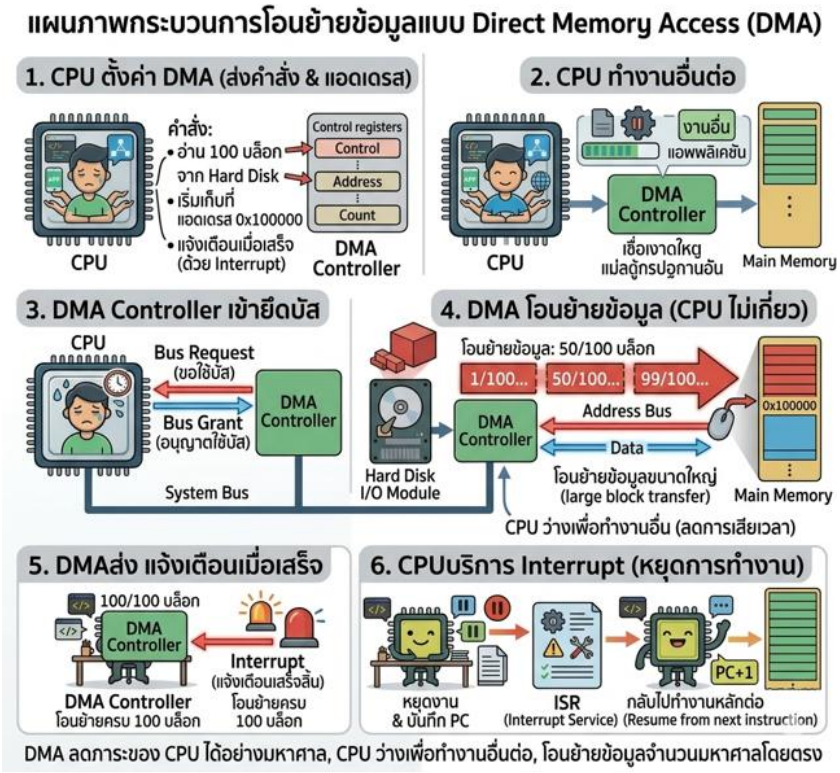
- เมื่อโปรแกรม (หรือระบบปฏิบัติการ) ต้องการโอนย้ายข้อมูลจำนวนมากจาก I/O Device ไปยัง Memory (หรือในทางกลับกัน) CPU จะเป็นผู้ **"ตั้งค่า" (Configure)** DMA Controller.
- CPU จะส่งข้อมูลที่จำเป็นให้กับ DMA Controller เช่น:
  - **แอดเดรสเริ่มต้น (Starting Address) ในหน่วยความจำ:** บอกว่าข้อมูลควรจะถูกอ่านหรือเขียนไปที่ตำแหน่งใดใน Main Memory.
  - **จำนวนไบต์ที่จะโอนย้าย (Number of Bytes to Transfer):** บอกว่าต้องการโอนย้ายข้อมูลเป็นปริมาณเท่าใด.
  - **ทิศทางการโอนย้าย (Direction of Transfer):** บอกว่าเป็นการอ่านจาก I/O ไป Memory หรือเขียนจาก Memory ไป I/O.
    - **แอดเดรสของพอร์ต I/O (I/O Port Address):** บอกว่าข้อมูลมาจากอุปกรณ์ I/O ตัวใด.
- หลังจากตั้งค่าเสร็จสิ้น CPU จะ **"สั่งให้ DMA Controller เริ่มทำงาน"**.

12.2.2 การโอนย้ายข้อมูล (Data Transfer) - CPU ฟรี (ให้ระบบอื่นควบคุมงานขนข้อมูล):

- เมื่อ DMA Controller ได้รับคำสั่งให้เริ่มทำงาน มันจะ **"ยึดครองบัส" (Bus Mastership)** จาก CPU ชั่วคราว.
- DMA Controller จะจัดการการโอนย้ายข้อมูลระหว่าง I/O Device และ Main Memory ด้วยตัวเอง โดยส่งสัญญาณแอดเดรสและสัญญาณควบคุมที่จำเป็นไปยัง Main Memory และ I/O Module.
- ในช่วงเวลานี้ **CPU จะเป็นอิสระ** สามารถไปประมวลผลคำสั่งหรือโปรแกรมอื่นได้ โดยไม่ต้องรอการโอนย้ายข้อมูล I/O.

12.2.3 การรับทราบเมื่อการโอนย้ายเสร็จสิ้น (Completion Notification):

- เมื่อ DMA Controller โอนย้ายข้อมูลครบตามจำนวนที่กำหนดแล้ว มันจะส่ง **"สัญญาณ Interrupt"** ไปยัง CPU.
- CPU จะรับรู้ Interrupt นี้ และดำเนินการ Interrupt Service Routine (ISR) ที่เกี่ยวข้อง เพื่อตรวจสอบว่าการโอนย้ายข้อมูลเสร็จสมบูรณ์หรือไม่ และจัดการกับข้อมูลที่โอนย้ายมา.
- หลังจากนั้น CPU ก็สามารถใช้ข้อมูลที่โอนย้ายมาได้ หรือสั่งการให้ DMA Controller ทำการโอนย้ายครั้งต่อไปได้.



รูปที่ 5-28 แผนภาพกระบวนการโอนย้ายข้อมูลแบบ DMA

12.3. การทำงานของ DMA และขั้นตอนเมื่อ CPU ต้องการอ่านหรือบันทึกข้อมูลจำนวนมาก

เช่น กรณี : CPU ต้องย้ายข้อมูลจำนวนมากจากฮาร์ดดิสก์ไปเก็บไว้ใน Main Memory

12.3.1 CPU ตั้งค่า DMA Controller:

- CPU เขียนคำสั่งไปยังรีจิสเตอร์ควบคุมของ DMA Controller (ซึ่งเป็นส่วนหนึ่งของ I/O Module หรือชิปเฉพาะ) เพื่อบอกว่า:
  - "อ่าน 100 บล็อกข้อมูลจากฮาร์ดดิสก์."
  - "เริ่มเก็บข้อมูลที่แอดเดรส 0x100000 ในหน่วยความจำ."
  - "เมื่อเสร็จแล้วให้แจ้งฉันด้วย Interrupt."

12.3.2 CPU เริ่มงานอื่น:

- หลังจากสั่งให้ DMA Controller เริ่มทำงาน CPU กลับไปประมวลผลงานอื่นๆ ที่มีอยู่

12.3.3 DMA Controller เข้ายึดบัส:

- DMA Controller ส่งสัญญาณ "Bus Request" ไปยัง CPU (หรือ Bus Arbiter) เพื่อขอใช้ System Bus.
- CPU (หรือ Bus Arbiter) ตอบกลับ "Bus Grant" เพื่ออนุญาต DMA Controller ใช้บัส.

12.3.4 DMA โอนย้ายข้อมูล (CPU ไม่เกี่ยว):

- DMA Controller ควบคุมการทำงานของฮาร์ดดิสก์ I/O Module ให้เริ่มอ่านข้อมูล.
- ฮาร์ดดิสก์อ่านข้อมูลมาทีละบล็อก.
- DMA Controller รับข้อมูลจากฮาร์ดดิสก์ผ่าน I/O Bus และส่งข้อมูลนั้นผ่าน Data Bus ไปยัง Main Memory โดยตรง.
- DMA Controller ยังคงส่งสัญญาณแอดเดรสที่ถูกต้องบน Address Bus ไปยัง Main Memory เพื่อระบุตำแหน่งที่จะเขียนข้อมูล.
- DMA Controller จะนับจำนวนข้อมูลที่โอนย้ายไปเรื่อยๆ จนกว่าจะครบ 100 บล็อก.

12.3.5 DMA ส่ง Interrupt เมื่อเสร็จ:

- เมื่อโอนย้ายข้อมูลครบ 100 บล็อกแล้ว DMA Controller จะส่งสัญญาณ Interrupt ไปยัง CPU.

12.3.6 CPU บริการ Interrupt: (หยุดการทำงาน)

- CPU รับ Interrupt, หยุดงานที่ทำอยู่ชั่วคราว, บันทึกสถานะ, และกระโดดไปทำ Interrupt Service Routine ที่เกี่ยวข้องกับ DMA.
- ISR จะตรวจสอบสถานะการโอนย้าย (สำเร็จ/ล้มเหลว) และสามารถแจ้งโปรแกรมว่าข้อมูลพร้อมใช้งานแล้ว.
- CPU กู้คืนสถานะและกลับไปทำงานหลักต่อ

ชุดข้อสอบเพื่อทบทวนความเข้าใจบทที่ 5 ไมโครโปรเซสเซอร์และการเชื่อมต่อระบบบัส

คำชี้แจง: เลือกคำตอบที่ถูกต้องที่สุดเพียงข้อเดียว (รวม 50 ข้อ)

1. วัตถุประสงค์หลักของ โครงสร้างการเชื่อมโยงภายในคอมพิวเตอร์ (Computer Interconnection Structure) คืออะไร?

   ก. เพื่อกำหนดสถาปัตยกรรมของชุดคำสั่ง (Instruction Set Architecture)

   ข. เพื่อจัดให้มีเส้นทางสำหรับการแลกเปลี่ยนข้อมูลระหว่างหน่วยประมวลผล หน่วยความจำ และ I/O

   ค. เพื่อจัดการการทำงานของระบบปฏิบัติการ (Operating System)

   ง. เพื่อควบคุมความเร็วของสัญญาณนาฬิกา (Clock Speed)

   เฉลย ข

2. ในโครงสร้างการเชื่อมต่อของระบบคอมพิวเตอร์ โมดูลใดที่ทำหน้าที่รับผิดชอบในการเชื่อมต่อและสื่อสารกับอุปกรณ์ภายนอกระบบ เช่น จอภาพและแป้นพิมพ์?

   ก. หน่วยประมวลผลกลาง (CPU)

   ข. หน่วยความจำหลัก (Main Memory)

   ค. โมดูล I/O (I/O Modules)

   ง. หน่วยควบคุม (Control Unit)

   เฉลย ค.

3. การเชื่อมต่อระหว่าง CPU, หน่วยความจำหลัก และ I/O Modules เป็นไปตามหลักการใด?

   ก. การเชื่อมต่อแบบจุดต่อจุด (Point-to-Point) สำหรับทุกโมดูล

   ข. การเชื่อมต่อทั้งหมดควบคุมโดย Control Unit ของ I/O

   ค. การเชื่อมต่อเกิดขึ้นผ่านระบบบัสเพื่อให้มีการแชร์เส้นทางการสื่อสารระหว่างส่วนประกอบต่าง ๆ

   ง. แต่ละโมดูลมีการใช้สายสัญญาณเฉพาะสำหรับตนเองโดยไม่แชร์ร่วมกัน

   เฉลย ค.

4. Data Bus มีหน้าที่หลักในการทำงานใด?

   ก. ใช้สำหรับระบุตำแหน่งที่อยู่ของหน่วยความจำหรือ I/O ที่ต้องการติดต่อ

   ข. ใช้ในการส่งสัญญาณควบคุมการทำงานไปยังอุปกรณ์ต่าง ๆ

   ค. ใช้ในการแจ้งเตือนการขัดจังหวะการทำงานของ CPU

   ง. ใช้ในการรับส่งข้อมูลระหว่างส่วนประกอบต่าง ๆ ในระบบ

   เฉลย ง.

5. Address Bus มีหน้าที่หลักในการทำงานใด?

   ก. ใช้สำหรับอ้างถึงตำแหน่งของหน่วยความจำหรือตำแหน่งพอร์ต I/O

   ข. ใช้ในการโอนย้ายข้อมูลคำสั่งเท่านั้น

   ค. ควบคุมทิศทางการไหลของข้อมูลบน Data Bus

   ง. เป็นช่องทางในการส่งสัญญาณ Interrupt

   เฉลย ก.

6. เมื่อ CPU ต้องการเขียนข้อมูลลงในหน่วยความจำ สัญญาณใดจาก Control Bus ที่ต้องทำงานร่วมกับ Address Bus และ Data Bus?

   ก. Interrupt Request

   ข. Memory Read

   ค. Memory Write

   ง. I/O Ready

   เฉลย ค.

7. Address Bus เชื่อมต่อกับอุปกรณ์หลักใดบ้าง เพื่อใช้ในการระบุตำแหน่งที่อยู่?

   ก. ALU และ Register ภายใน CPU เท่านั้น

   ข. Data Bus และ Control Bus เท่านั้น

   ค. อุปกรณ์ต่อพ่วงภายนอก (Peripheral Devices) เท่านั้น

   ง. CPU, หน่วยความจำหลัก และ I/O Module

   เฉลย ง.

8. เมื่อ CPU ต้องการอ่านข้อมูลจาก I/O Module, CPU จะต้องส่งสัญญาณใดผ่าน Control Bus เพื่อสื่อสารกับ I/O Module นั้น?

   ก. Memory Write ข. I/O Write

   ค. Memory Read ง. I/O Read

   เฉลย ง.

9. การทำงานของ CPU Bus โดยรวมมีวัตถุประสงค์เพื่ออะไร?

   ก. เป็นช่องทางเฉพาะในการเชื่อมต่อระหว่าง CPU กับ Cache Memory เท่านั้น

   ข. เป็นช่องทางหลักในการสื่อสารระหว่าง CPU กับโมดูลหลักอื่น ๆ เพื่อดึงคำสั่งและโอนย้ายข้อมูล

   ค. ควบคุมการจ่ายพลังงานไฟฟ้าให้กับทุกส่วนประกอบในระบบ

   ง. จัดการการทำงานของระบบปฏิบัติการ

   เฉลย ข.

10. ในขั้นตอน Instruction Fetch (ดึงคำสั่ง) CPU ใช้ Address Bus เพื่อส่งข้อมูลใดไปยังหน่วยความจำ?

    ก. ข้อมูลของคำสั่ง (Instruction Data)

    ข. ผลลัพธ์จากการประมวลผล (Result)

    ค. ตำแหน่งที่อยู่ของคำสั่งถัดไปที่ต้องถูกดึง (Address of the next instruction)

    ง. สัญญาณควบคุม Write

    เฉลย ค.

11. หน้าที่ของ CPU Bus ในส่วนของ Data Access คืออะไร?

    ก. เป็นช่องทางให้ CPU สามารถอ่านหรือเขียนข้อมูลไปยังหน่วยความจำหรือ I/O Module

    ข. ใช้ในการส่งสัญญาณควบคุมการทำงานไปยังอุปกรณ์ภายนอก

    ค. ใช้ในการแจ้งเตือนการขัดจังหวะการทำงานของ CPU

    ง. ใช้ในการดึงคำสั่งจากหน่วยความจำหลัก

    เฉลย ก.

12. สัญญาณใดบน CPU Bus ที่เกี่ยวข้องกับ Interrupt Handling และทำหน้าที่อนุญาตให้ I/O Module แจ้ง CPU ว่าต้องการความช่วยเหลือ?

    ก. Control Signals ข. Address Signals

    ค. Data Signals ง. Interrupt Signals

    เฉลย ง.

13. สัญญาณใดบน CPU Bus ที่ทำหน้าที่บอกว่ามีอุปกรณ์ภายนอกต้องการติดต่อสื่อสารกับ CPU?

    ก. Instruction Signals

    ข. Interrupt Signals

    ค. Control Signals

    ง. Address Signals

    เฉลย ข.

14. ในบริบทของ CPU Bus, Control Signals มีตัวอย่างการทำงานอย่างไร?

    ก. ระบุตำแหน่งหน่วยความจำที่ CPU กำลังเข้าถึง

    ข. เป็นตัวข้อมูลที่จะถูกโอนย้าย

    ค. สัญญาณ Read หรือ Write เพื่อร้องขอการเข้าถึงหน่วยความจำ

    ง. ชุดคำสั่งที่ถูกดึงมาจากหน่วยความจำ

    เฉลย ค.

15. ความแตกต่างหลักระหว่าง Instruction และ Data บน CPU Bus คืออะไร?

    ก. Instruction ถูกส่งผ่าน Control Bus ส่วน Data ถูกส่งผ่าน Data Bus

    ข. Instruction ใช้สำหรับจัดเก็บข้อมูล ส่วน Data ใช้สำหรับประมวลผล

    ค. Instruction คือชุดคำสั่งที่บอกให้ทำอะไร ส่วน Data คือข้อมูลที่ใช้สำหรับโอนย้าย

    ง. Instruction ถูกเข้ารหัสแบบไบนารี ส่วน Data ถูกเข้ารหัสแบบ BCD

    เฉลย ค.

16. หน้าที่หลักของ Memory Bus คืออะไร?

    ก. เป็นช่องทางเฉพาะสำหรับการรับส่งข้อมูลระหว่าง I/O Module และ I/O Device เท่านั้น

    ข. เป็นช่องทางเฉพาะสำหรับการรับส่งข้อมูลระหว่าง CPU และหน่วยความจำหลัก

    ค. ใช้ในการควบคุมลำดับการทำงานของ ALU

    ง. จัดการการแลกเปลี่ยนข้อมูลระหว่าง I/O Device กับหน่วยความจำหลักโดยตรง

    เฉลย ข.

17. สัญญาณใดบน Memory Bus ที่บ่งบอกว่า CPU ต้องการเขียนข้อมูลไปยังหน่วยความจำหลัก?

    ก. Read ข. Address

    ค. Data ง. Write

    เฉลย ง.

18. เมื่อ CPU ใช้ Memory Bus เพื่อเข้าถึงข้อมูลในหน่วยความจำ Address และ Data มีบทบาทอย่างไรในการดำเนินการ "Read"?

    ก. Address ระบุข้อมูลที่จะอ่าน และ Data ระบุตำแหน่งของข้อมูล

    ข. Address และ Data ถูกใช้เพื่อบอกว่ามีการเขียนข้อมูลเท่านั้น

    ค. Address ระบุตำแหน่งที่ต้องการอ่านข้อมูล และ Data คือข้อมูลที่ถูกอ่านจากตำแหน่งนั้น

    ง. Address ระบุสัญญาณควบคุม และ Data ระบุเวลาที่ใช้ในการเข้าถึง

    เฉลย ค.

19. การทำงานของ I/O Bus System มีความแตกต่างจาก Memory Bus อย่างไร?

    ก. I/O Bus มีเพียง Address Bus เท่านั้น

    ข. I/O Bus ใช้ในการโอนย้ายข้อมูลจำนวนมากเท่านั้น

    ค. I/O Bus ไม่ต้องใช้สัญญาณควบคุม (Control Signals)

    ง. I/O Bus มีสัญญาณ Interrupt Signals เพิ่มเติมเพื่อใช้แจ้ง CPU เมื่อ I/O ต้องการติดต่อ

    เฉลย ง.

20. ในการโอนย้ายข้อมูลระหว่าง System Bus กับ I/O Module Device I/O Module มีบทบาทสำคัญที่สุดอย่างไร?

    ก. ทำหน้าที่แปลงข้อมูลที่ส่งผ่าน System Bus ให้อยู่ในรูปแบบที่ I/O Device สามารถทำงานได้

    ข. I/O Module เป็นเพียงช่องทางที่ไม่มีการประมวลผลใด ๆ

    ค. I/O Module ควบคุมความเร็วในการประมวลผลของ CPU

    ง. I/O Module สามารถตัดสินใจได้เองว่าจะใช้ DMA หรือไม่

    เฉลย ก.

21. พอร์ต (Port) ในบริบทของ I/O มีหน้าที่สำคัญอย่างไร?

    ก. เป็นหน่วยความจำสำรองสำหรับ I/O Device

    ข. เป็นช่องทางในการติดต่อกับ I/O Module โดยเชื่อมต่อผ่านทางระบบบัส

    ค. ใช้สำหรับจัดเก็บชุดคำสั่งของ I/O Device

    ง. ทำหน้าที่เป็นตัวควบคุมสัญญาณนาฬิกา

22. การโอนย้ายข้อมูลผ่านพอร์ตโดยทั่วไปจะใช้ I/O Address เพื่อวัตถุประสงค์ใด?

    ก. เพื่อระบุความเร็วในการโอนย้ายข้อมูล

    ข. เพื่อจัดเก็บข้อมูลชั่วคราวขณะโอนย้าย

    ค. เพื่อระบุตำแหน่งที่อยู่เฉพาะของพอร์ต I/O ที่ CPU ต้องการติดต่อ

    ง. เพื่อบอกประเภทของ I/O Device ที่เชื่อมต่ออยู่

23. บทบาทของ System Bus ในการโอนย้ายข้อมูลไปยัง พอร์ต คือการส่งสัญญาณใดบ้าง?

    ก. สัญญาณนาฬิกาเท่านั้น

    ข. สัญญาณอินเทอร์รัปต์เท่านั้น

    ค. สัญญาณควบคุมการจ่ายไฟเท่านั้น

    ง. สัญญาณแอดเดรส ข้อมูล และควบคุม เพื่อเข้าถึงและควบคุมพอร์ต I/O

    เฉลย ง.

24. เทคนิคใดในการโอนย้ายข้อมูลที่ CPU ต้องตรวจสอบสถานะของ I/O Module อย่างต่อเนื่อง (Polling) ทำให้ CPU มีภาระงานสูง?

    ก. Programmed I/O

    ข. Interrupt-Driven I/O

    ค. Direct Memory Access (DMA)

    ง. Memory-Mapped I/O

    เฉลย ก.

25. เทคนิค Interrupt-Driven I/O ช่วยแก้ปัญหาภาระงานของ CPU ในการจัดการ I/O เมื่อเทียบกับ Programmed I/O ได้อย่างไร?

    ก. ทำให้ I/O Device โอนย้ายข้อมูลกับหน่วยความจำโดยตรง

    ข. CPU ไม่ต้องจัดการ I/O ใด ๆ อีกต่อไป

    ค. I/O Module แจ้ง CPU เมื่อพร้อมโอนย้ายข้อมูล ทำให้ CPU สามารถไปทำงานอื่นได้ในระหว่างรอ

    ง. ลดความจำเป็นในการใช้ Control Bus

    เฉลย ค.

26. การโอนย้ายข้อมูลจำนวนน้อย ๆ ที่ใช้เวลาสั้น ๆ มักใช้เทคนิคใดเพื่อให้ง่ายต่อการจัดการ?

    ก. Direct Memory Access (DMA)
    ข. Programmed I/O
    ค. Vectored Interrupt I/O
    ง. Nonmaskable Interrupt I/O

    เฉลย ข.

27. เทคนิคใดที่เหมาะสมที่สุดสำหรับการถ่ายโอนข้อมูลระหว่าง I/O Device กับหน่วยความจำหลักโดยมีจุดประสงค์เพื่อ ลดภาระ CPU อย่างมีนัยสำคัญ?

    ก. Programmed I/O
    ข. Interrupt-Driven I/O
    ค. Direct Memory Access (DMA)
    ง. Memory-Mapped I/O

    เฉลย ค.

28. หน้าที่หลักของ Interrupt Controller คืออะไร?

    ก. จัดการสัญญาณ Interrupt จากหลายอุปกรณ์ และส่งสัญญาณเดียวไปยัง CPU

    ข. เป็นตัวควบคุมการจ่ายไฟให้กับ I/O Module

    ค. ทำหน้าที่คำนวณตำแหน่งแอดเดรสของหน่วยความจำ

    ง. ควบคุมการทำงานของ Data Bus และ Address Bus โดยตรง

29. เมื่อ Interrupt Controller ได้รับสัญญาณ Interrupt จากหลายอุปกรณ์พร้อมกัน สิ่งที่ต้องทำเป็นอันดับแรกคืออะไร?

    ก. ส่งสัญญาณทั้งหมดไปยัง CPU โดยไม่มีการจัดการ

    ข. ทำการจัดลำดับความสำคัญ (Priority Handling) ของสัญญาณ Interrupt

    ค. ส่งสัญญาณไปที่ I/O Device ที่มีความเร็วสูงสุดก่อน

    ง. รอให้ CPU ว่างก่อนจึงจะส่งสัญญาณ

    เฉลย ข.

30. Nonmaskable Interrupt (NMI) มีลักษณะเด่นอย่างไร?

ก. เป็น Interrupt ที่เกิดจากซอฟต์แวร์เท่านั้น

ข. เป็น Interrupt ที่สามารถถูกละเลยหรือถูกปิดการทำงานได้โดย CPU

ค. มีลำดับความสำคัญต่ำที่สุดในระบบ

ง. เป็น Interrupt ที่มีลำดับความสำคัญสูงที่สุดและ CPU ไม่สามารถเพิกเฉยหรือปิดการทำงานได้

เฉลย ง.

31. Maskable Interrupt (MI) คือ Interrupt ประเภทใด?

ก. Interrupt ที่ใช้สำหรับแจ้งข้อผิดพลาดร้ายแรงของระบบ

ข. Interrupt ที่ต้องใช้ DMA ในการโอนย้ายข้อมูล

ค. Interrupt ที่เกิดขึ้นจากสัญญาณนาฬิกาเท่านั้น

ง. Interrupt ที่ CPU สามารถปิดการทำงาน (Disable) ได้หากต้องการ

เฉลย ง.

32. Hardware Interrupt เกิดขึ้นจากสาเหตุใด?

ก. เกิดจากการประมวลผลคำสั่งพิเศษในโปรแกรม

ข. เกิดขึ้นเมื่อ CPU ต้องการเปลี่ยนโปรแกรมที่ทำงาน

ค. เกิดจากอุปกรณ์ภายนอก เช่น การกดแป้นพิมพ์ หรือ I/O Device ต้องการบริการจาก CPU

ง. เกิดขึ้นเมื่อเกิดข้อผิดพลาดในการคำนวณทางคณิตศาสตร์

เฉลย ค.

33. Software Interrupt แตกต่างจาก Hardware Interrupt อย่างไร?

ก. ถูกเรียกโดยโปรแกรม (คำสั่ง) ที่กำลังทำงานอยู่ เพื่อร้องขอบริการจากระบบปฏิบัติการ

ข. ถูกจัดลำดับความสำคัญต่ำกว่าเสมอ

ค. ไม่ต้องใช้ Interrupt Controller

ง. ไม่สามารถหยุดการทำงานของโปรแกรมหลักได้

เฉลย ก.

34. ตัวอย่างของ Hardware Interrupt คืออะไร?

ก. การเรียกใช้ฟังก์ชัน 'system call' ในโปรแกรม

ข. การหารด้วยศูนย์ (Divide by Zero)

ค. สัญญาณจากเมาส์คลิกเพื่อขอให้ CPU ประมวลผล

ง. การใช้คำสั่ง 'trap' ในโปรแกรม

เฉลย ค.

35. ในระบบ Vectored Interrupt ข้อมูลเพิ่มเติมใดที่ I/O Device ส่งให้กับ CPU เพื่อระบุที่อยู่ของ Routine ที่ใช้ในการจัดการ Interrupt?

    ก. Address ของ CPU Register

    ข. สถานะของ Control Bus

    ค. Interrupt Vector (ที่เก็บตำแหน่งของ Interrupt Service Routine)

    ง. ขนาดของ Data Bus

    เฉลย ค.

36. ข้อจำกัดหลักของ Non-Vectored Interrupt คืออะไร?

    ก. I/O Device ทุกตัวจะแชร์ Interrupt Service Routine (ISR) เดียวกัน

    ข. CPU ต้องทำการ Polling I/O Module ทุกตัวเพื่อตรวจสอบแหล่งที่มา

    ค. ไม่สามารถใช้กับ Hardware Interrupt ได้

    ง. มีขั้นตอนการทำงานที่เร็วกว่า Vectored Interrupt

    เฉลย ข.

37. ในขั้นตอนแรกของ Vectored Interrupt หลังจากที่ CPU ได้รับสัญญาณ Interrupt แล้ว สิ่งที่ CPU ต้องทำก่อนกระโดดไปทำ ISR คืออะไร?

    ก. สั่งให้ DMA Controller เริ่มทำงาน

    ข. อ่านข้อมูลจาก Address Bus

    ค. ส่งสัญญาณ I/O Read ไปยัง I/O Device

    ง. เก็บตำแหน่งถัดไปของโปรแกรมหลักลงบนหน่วยความจำของสแตก

    เฉลย ง.

38. การทำงานของ Vectored Interrupt สรุปได้ว่าอย่างไร?

    ก. CPU หยุดทำงานโดยสมบูรณ์

    ข. CPU หยุดทำงานที่กำลังประมวลผลชั่วคราว กระโดดไปทำโปรแกรมที่มีการอินเทอร์รัปต์เข้ามา

    ค. I/O Device ทำงานโดยไม่ผ่าน CPU

    ง. CPU ทำงานต่อโดยไม่สนใจ Interrupt

    เฉลย ข.

39. การโอนย้ายข้อมูลแบบ DMA คือวิธีการโอนย้ายข้อมูลอย่างไร?

    ก. โอนย้ายข้อมูลระหว่าง CPU กับหน่วยความจำ

    ข. โอนย้ายข้อมูลระหว่าง I/O Device กับ CPU

    ค. โอนย้ายข้อมูลโดยไม่ผ่าน CPU ซึ่ง I/O Module โอนถ่ายข้อมูลให้กับหน่วยความจำโดยตรง

ง. โอนย้ายข้อมูลระหว่าง Register ภายใน CPU เท่านั้น

เฉลย ค.

40. หน้าที่หลักของ DMA Controller ที่เกี่ยวข้องกับ System Bus คืออะไร?

ก. แปลงสัญญาณดิจิทัลเป็นอนาล็อก

ข. ทำหน้าที่ควบคุม System Bus (Address, Data, Control Bus) ชั่วคราว ในช่วงที่มีการโอนย้ายข้อมูลระหว่างหน่วยความจำและ I/O

ค. จัดเก็บ Interrupt Vector

ง. กำหนดลำดับความสำคัญของ CPU Register

เฉลย ข.

41. ก่อนที่ DMA Controller จะเริ่มโอนย้ายข้อมูลกับหน่วยความจำโดยตรง CPU ต้องทำอะไรก่อน?

ก. CPU ต้องโอนย้ายข้อมูลไปทั้งหมดก่อน

ข. CPU ต้องส่งข้อมูลเกี่ยวกับจำนวนข้อมูลและแอดเดรสที่จะโอนย้ายไปยัง DMA Controller

ค. CPU ต้องสั่งให้ I/O Device หยุดทำงาน

ง. CPU ต้องส่งสัญญาณ NMI

เฉลย ข.

42. DMA Controller เข้ามาควบคุม System Bus ชั่วคราวในลักษณะใด?

ก. ส่งสัญญาณ Interrupt ไปที่ CPU

ข. ร้องขอการควบคุมบัส (Bus Request) และได้รับสัญญาณอนุญาต (Bus Grant) จาก CPU

ค. บังคับให้ Address Bus หยุดทำงาน

ง. เปลี่ยนความเร็วของ Data Bus

เฉลย ข.

43. อุปกรณ์ I/O ใดที่เหมาะสมที่สุดสำหรับการใช้กระบวนการ DMA?

ก. เมาส์ ข. แป้นพิมพ์

ค. ฮาร์ดดิสก์ (Hard Disk)) ง. ลำโพง

เฉลย ค.

44. การทำงานของ DMA ช่วยเพิ่มประสิทธิภาพของระบบโดยรวมในกรณีของฮาร์ดดิสก์ได้อย่างไร?

ก. ทำให้ฮาร์ดดิสก์ทำงานเร็วขึ้นสองเท่า

ข. ลดภาระงานของ CPU เพราะ CPU ไม่ต้องเสียเวลาจัดการข้อมูลทีละเล็กน้อยในระหว่างการโอนย้ายข้อมูลจำนวนมาก

ค. ทำให้ CPU ไม่ต้องติดต่อกับหน่วยความจำหลัก

ง. ลดความจำเป็นในการใช้ System Bus

เฉลย ข.

45. หลังจากที่ DMA Controller ได้รับคำสั่งจาก CPU และดำเนินการโอนย้ายข้อมูลเสร็จสมบูรณ์แล้ว DMA Controller จะแจ้ง CPU อย่างไร?

ก. ส่งสัญญาณ Hold ไปที่หน่วยความจำ

ข. ส่งสัญญาณ Read Request ไปที่ I/O Device

ค. ส่งสัญญาณ Interrupt กลับไปยัง CPU เพื่อแจ้งว่าทำงานเสร็จสิ้น

ง. ปิดการทำงานของ Address Bus

เฉลย ค.

46. ในระหว่างที่ DMA Controller กำลังโอนย้ายข้อมูลระหว่างฮาร์ดดิสก์กับหน่วยความจำหลัก, CPU สามารถทำอะไรได้?

ก. หยุดการทำงานโดยสมบูรณ์

ข. ว่างและสามารถทำงานประมวลผลอื่น ๆ (เช่น รันโปรแกรมของผู้ใช้) ได้

ค. ต้องรอการโอนย้ายข้อมูลให้เสร็จสิ้นเท่านั้น

ง. ควบคุมการจ่ายไฟไปยัง I/O Module

เฉลย ข.

47. หาก System Bus มีขนาด 32 บิตใน Data Bus และ Address Bus ข้อใดคือข้อสรุปที่ถูกต้องที่สุด?

ก. CPU สามารถโอนย้ายข้อมูลได้ครั้งละ 64 บิต

ข. CPU สามารถติดต่อกับ I/O ได้ไม่เกิน 2^32 พอร์ต

ค. CPU สามารถอ้างตำแหน่งหน่วยความจำได้สูงสุด 2^32 ตำแหน่ง และโอนย้ายข้อมูลได้ครั้งละ 32 บิต

ง. System Bus มีทั้งหมด 64 เส้นสัญญาณ

เฉลย ค.

48. Programmed I/O ต่างจาก Interrupt-Driven I/O ในประเด็นหลักใด?

ก. Programmed I/O ใช้ DMA แต่ Interrupt-Driven I/O ไม่ใช้

ข. Interrupt-Driven I/O ทำให้ CPU ต้อง Polling สถานะ I/O บ่อยกว่า

ค. Programmed I/O ทำให้ CPU ต้องเสียเวลาไปกับการรอ I/O แต่ Interrupt-Driven I/O ให้ I/O เป็นฝ่ายแจ้ง

ง. Programmed I/O ไม่สามารถจัดการการโอนย้ายข้อมูลได้

เฉลย ค.

49. หากเกิด Interrupt พร้อมกันสองเหตุการณ์คือ A (Maskable Interrupt) และ B (Nonmaskable Interrupt) CPU ควรดำเนินการอย่างไรตามลำดับความสำคัญ?

    ก. CPU ต้องตอบสนองต่อ B (NMI) ทันที เนื่องจากไม่สามารถเพิกเฉยได้ และค่อยจัดการ A (MI)

    ข. CPU ตอบสนองต่อ A (MI) ก่อน เนื่องจากมีลำดับความสำคัญในการทำงานปกติ

    ค. CPU ส่งสัญญาณ Polling ไปที่ทั้งสองอุปกรณ์เพื่อตัดสินใจ

    ง. CPU เพิกเฉยต่อทั้งสอง Interrupt และทำงานต่อไป

    เฉลย ก.

50. ข้อดีหลักของ Vectored Interrupt เมื่อเทียบกับ Non-Vectored Interrupt คืออะไร?

    ก. สามารถจัดการเฉพาะ Hardware Interrupt เท่านั้น

    ข. มีขั้นตอนการทำงานที่ซับซ้อนน้อยกว่า

    ค. ลดเวลาที่ใช้ในการระบุแหล่งที่มาของ Interrupt เนื่องจากมีเวกเตอร์ระบุตำแหน่ง ISR โดยตรง

    ง. ไม่ต้องใช้ Interrupt Controller

    เฉลย ค.